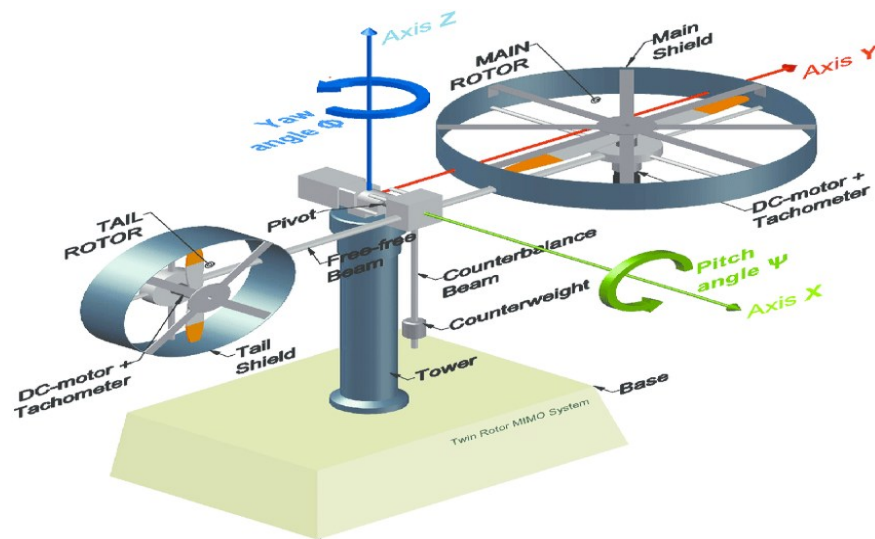




LABORATORY MANUAL

for

ECE2003- Control Systems Engineering

Programme Name & Branch: B. Tech ECE



**School of Electronics Engineering
VIT-AP University, Amaravati-522241.**

PROGRAMME EDUCATIONAL OBJECTIVES (PEO's)

- PEO1:** Have Professional competency through the application of knowledge gained from subjects like Mathematics, Physics, Chemistry, Inter-Disciplinary and core subjects like Signal Processing, VLSI, Embedded Systems, Communication and Automation. **(Professional Competency)**
- PEO2:** *Excel* in one's career by critical thinking towards successful services and growth of the organization or as an entrepreneur or through higher studies. **(Successful Career Goals)**
- PEO3:** Enhance knowledge by updating advanced technological concepts for facing the rapidly changing world and contribute to society through innovation and creativity. **(Continuing Education and Contribution to Society)**

PROGRAMME OUTCOMES (PO's)

- PO1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO2. **Problem analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

- PO5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- PO6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- PO11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Expt. No.:1

Date:

Assessment of time-domain performance indices from the transfer function of a system using MATLAB

- **AIM/OBJECTIVE:** Plot unit step response of given transfer function and finds delay time, rise time, peak time and peak overshoot (time domain specifications).
- **APPARATUS /COMPONENTS REQUIRED: (For Hardware Lab)**

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

- **THEORY/METHODOLOGY:**

The time response represents how the state of a dynamic system changes in time when subjected to a particular input. Since the models we have derived consist of differential equations, some integration must be performed in order to determine the time response of the system. For some simple systems, a closed-form analytical solution may be available. However, for most systems, especially nonlinear systems or those subject to complicated inputs, this integration must be carried out numerically. Fortunately, MATLAB provides many useful resources for calculating time responses for many types of inputs, as we shall see in the following sections.

The time response of a linear dynamic system consists of the sum of the transient response which depends on the initial conditions and the steady-state response which depends on the system input. These correspond to the homogenous (free or zero input) and the particular solutions of the governing differential equations, respectively.

(A) First-Order Systems

First-order systems are the simplest dynamic systems to analyze. Some common examples include mass-damper systems and RC circuits.

The general form of the first-order differential equation is as follows

$$\dot{y} + ay = bu \quad \text{or} \quad \tau \dot{y} + y = k_{dc}u \quad (1)$$

The form of a first-order transfer function is

$$G(s) = \frac{b}{s + a} = \frac{k_{dc}}{\tau s + 1} \quad (2)$$

where the parameters k_{dc} and τ completely define the character of the first-order system.

DC Gain

The DC gain, k_{dc} is the ratio of the magnitude of the steady-state step response to the magnitude of the step input. For stable transfer functions, the **Final Value Theorem** demonstrates that the DC gain is the value of the transfer function evaluated at $s = 0$. For first-order systems of the forms shown, the DC gain is .

$$k_{dc} = b/a$$

Time Constant

The time constant of a first-order system $T_c = \tau = 1/a$ is which is equal to the time it takes for the system's response to reach 63% of its steady-state value for a step input (from zero initial conditions) or to decrease to 37% of the initial value for a system's free response. More generally, it represents the time scale for which the dynamics of the system are significant.

Poles/Zeros

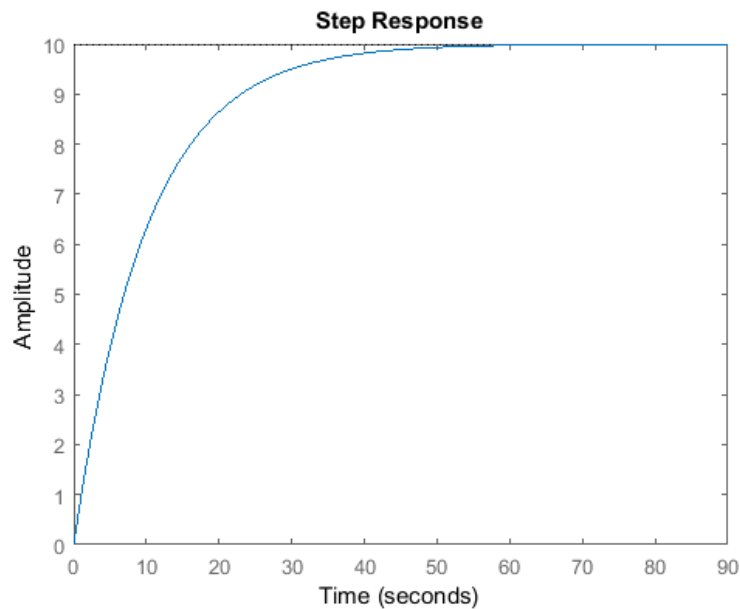
First-order systems have a single real pole, in this case at $s = -a$. Therefore, the system is stable if a is positive and unstable if a is negative. Standard first-order system have no zeros.

Step Response

We can calculate the system time response to a step input of magnitude u using the following MATLAB commands:

```
k_dc = 5;
Tc = 10;
u = 2;
s = tf('s');
G = k_dc/(Tc*s+1)
step(u*G)
Output:
```

$$G = \frac{5}{10s + 1}$$



If you right-click on the step response graph and select **Characteristics**, you can choose to have several system metrics overlaid on the response: peak response, settling time, rise time, and steady-state.

Settling Time

The settling time, T_s , is the time required for the system output to fall within a certain percentage (i.e. 2%) of the steady-state value for a step input. The settling times for a first-order system for the most common tolerances are provided in the table below. Note that the tighter the tolerance, the longer the system response takes to settle to within this band, as expected.

10%	5%	2%	1%
$T_s=2.3/a=2.3T_c$	$T_s=3/a=3T_c$	$T_s=3.9/a=3.9T_c$	$T_s=4.6/a=4.6T_c$

Rise Time

The rise time, T_r , is the time required for the system output to rise from some lower level $x\%$ to some higher level $y\%$ of the final steady-state value. For first-order systems, the typical range is 10% - 90%.

(B) Second-Order Systems

Second-order systems are commonly encountered in practice, and are the simplest type of dynamic system to exhibit oscillations. Examples include mass-spring-damper systems and RLC circuits. In fact, many true higher-order systems may be approximated as second-order in order to facilitate analysis.

The canonical form of the second-order differential equation is as follows

$$m\ddot{y} + b\dot{y} + ky = f(t) \quad \text{or} \quad \ddot{y} + 2\zeta\omega_n\dot{y} + \omega_n^2 y = k_{dc}\omega_n^2 u \quad (4)$$

The canonical second-order transfer function has the following form, in which it has two poles and no zeros.

$$G(s) = \frac{1}{ms^2 + bs + k} = \frac{k_{dc}\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2} \quad (5)$$

The parameters k_{dc} , ζ , and ω_n characterize the behavior of a canonical second-order system.

DC Gain

The DC gain, k_{dc} , again is the ratio of the magnitude of the steady-state step response to the magnitude of the step input, and for stable systems it is the value of the transfer function when $s = 0$. For the forms given,

$$k_{dc} = \frac{1}{k} \quad (6)$$

Damping Ratio

The damping ratio ζ is a dimensionless quantity characterizing the rate at which an oscillation in the system's response decays due to effects such as viscous friction or electrical resistance. From the above definitions,

$$\zeta = \frac{b}{2\sqrt{km}} \quad (7)$$

Natural Frequency

The natural frequency ω_n is the frequency (in rad/s) that the system will oscillate at when there is no damping, $\zeta = 0$.

$$\omega_n = \sqrt{\frac{k}{m}} \quad (8)$$

Poles/Zeros

The canonical second-order transfer function has two poles at:

$$s_p = -\zeta\omega_n \pm j\omega_n\sqrt{1 - \zeta^2} \quad (9)$$

Underdamped Systems

If $\zeta < 1$, then the system is **underdamped**. In this case, both poles are complex-valued with negative real parts; therefore, the system is stable but oscillates while approaching the steady-state value. Specifically, the natural response oscillates with the damped natural

frequency, $\omega_d = \omega_n\sqrt{1 - \zeta^2}$ (in rad/sec).

k_dc = 1;

w_n = 10;

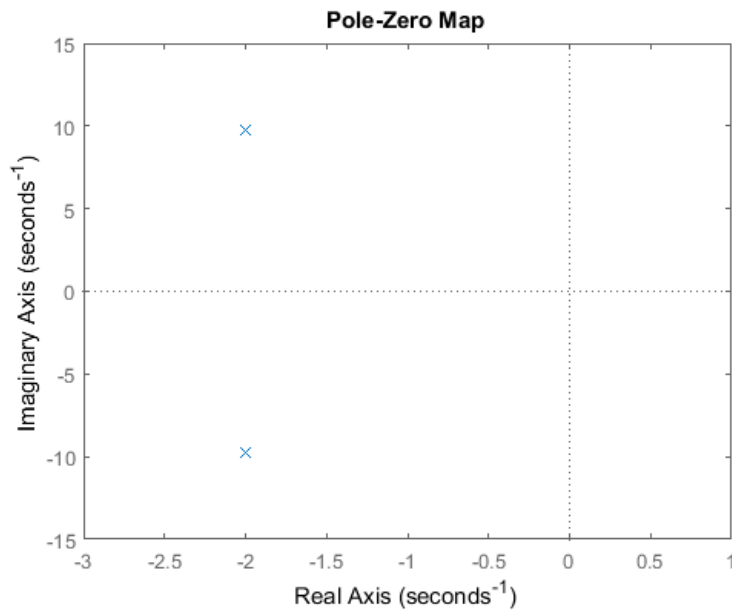
zeta = 0.2;

s = tf('s');

G1 = k_dc*w_n^2/(s^2 + 2*zeta*w_n*s + w_n^2);

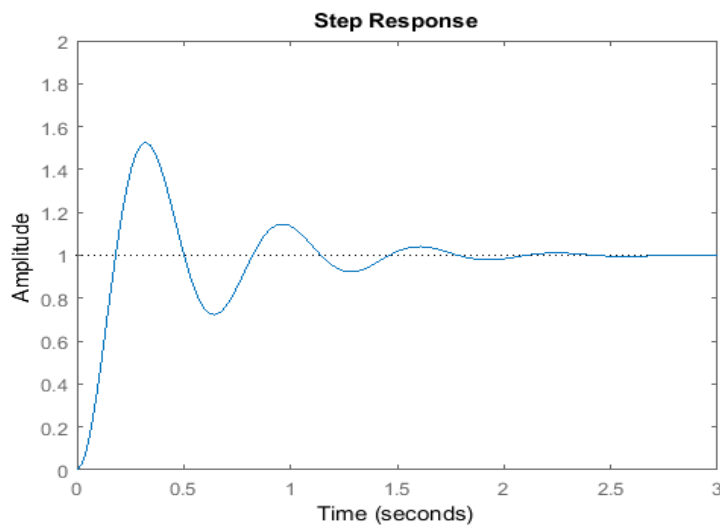
pzmap(G1)

axis([-3 1 -15 15])



`step(G1)`

`axis([0 3 0 2])`



Settling Time: The settling time, T_s , is the time required for the system output to fall within a certain percentage of the steady-state value for a step input. For a canonical second-order, underdamped system, the settling time can be approximated by the following equation:

$$T_s = \frac{-\ln(\text{tolerance fraction})}{\zeta\omega_n} \quad (10)$$

The settling times for the most common tolerances are presented in the following table:

10%	5%	2%	1%
$T_s=2.3/(\zeta\omega_n)$	$T_s=3/(\zeta\omega_n)$	$T_s=3.9/(\zeta\omega_n)$	$T_s=4.6/(\zeta\omega_n)$

Percent Overshoot

The percent overshoot is the percent by which a system's step response exceeds its final steady-state value. For a second-order underdamped system, the percent overshoot M_p is directly related to the damping ratio by the following equation. Here, M_p is a decimal number where 1 corresponds to 100% overshoot.

$$M_p = e^{\left(\frac{-\zeta\pi}{\sqrt{1-\zeta^2}}\right)} \quad (11)$$

For second-order underdamped systems, the 1% settling time, T_s , 10-90% rise time, T_r , and percent overshoot, M_p , are related to the damping ratio and natural frequency as shown below.

$$T_s \approx \frac{4.6}{\zeta\omega_n} \quad (12)$$

$$T_r \approx \frac{1.8}{\omega_n} \quad (13)$$

$$\zeta = \frac{-\ln(M_p)}{\sqrt{\pi^2 + \ln^2(M_p)}} \quad (14)$$

Overdamped Systems

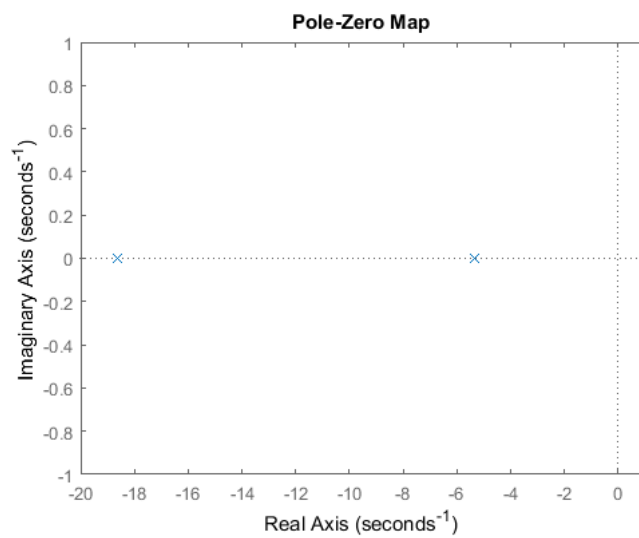
If $\zeta > 1$, then the system is **overdamped**. Both poles are real and negative; therefore, the system is stable and does not oscillate. The step response and a pole-zero map of an overdamped system are calculated below:

```
zeta = 1.2;
```

```
G2 = k_dc*w_n^2/(s^2 + 2*zeta*w_n*s + w_n^2);
```

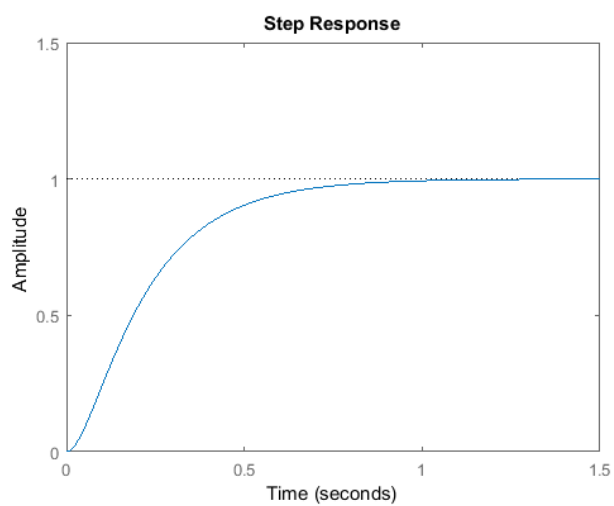
```
pzmap(G2)
```

```
axis([-20 1 -1 1])
```



```
step(G2)
```

```
axis([0 1.5 0 1.5])
```



Critically-Damped Systems

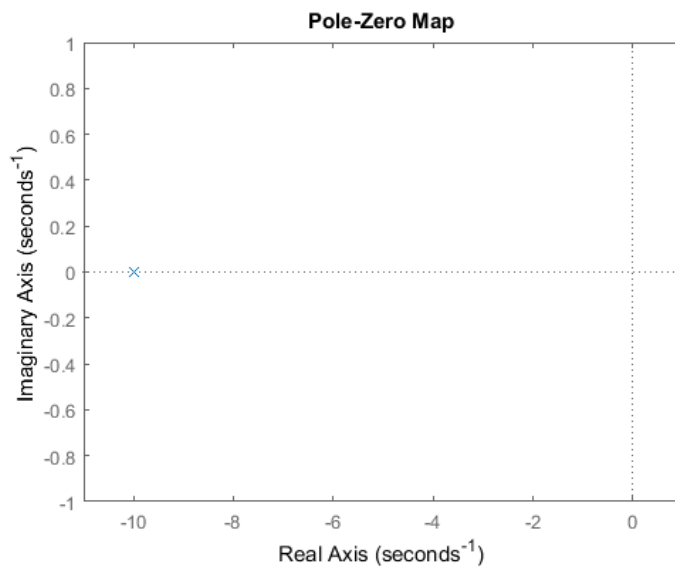
If $\zeta = 1$, then the system is **critically damped**. Both poles are real and have the same magnitude, $s_p = -\zeta\omega_n$. For a canonical second-order system, the quickest settling time is achieved when the system is critically damped. Now change the value of the damping ratio to 1, and re-plot the step response and pole-zero map.

```
zeta = 1;
```

```
G3 = k_dc*w_n^2/(s^2 + 2*zeta*w_n*s + w_n^2);
```

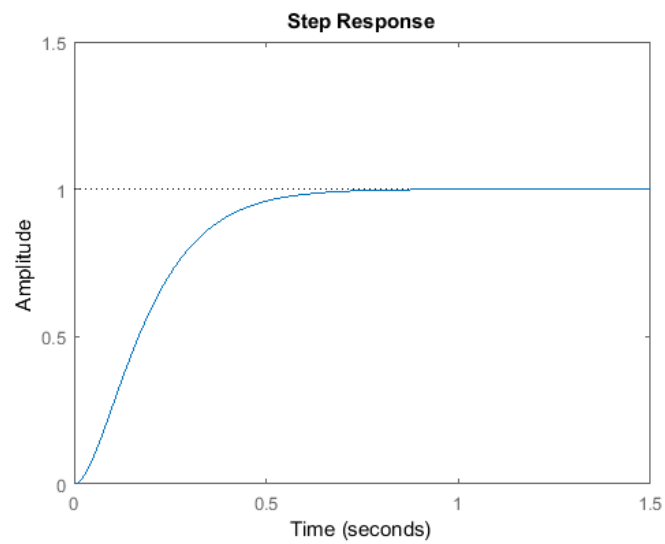
```
pzmap(G3)
```

```
axis([-11 1 -1 1])
```



```
step(G3)
```

```
axis([0 1.5 0 1.5])
```



Undamped Systems

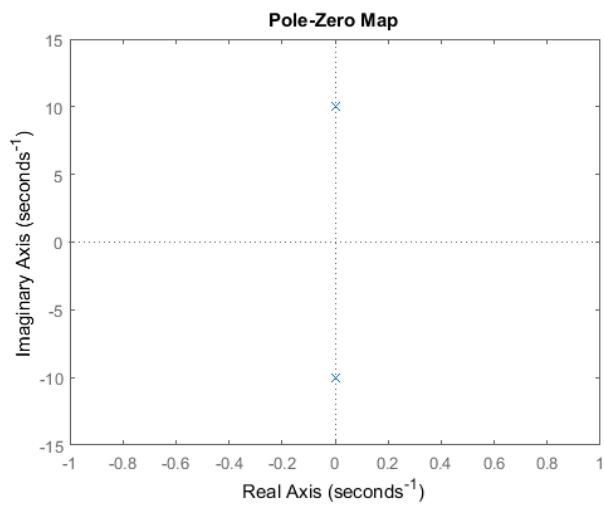
If $\zeta = 0$, then the system is **undamped**. In this case, the poles are purely imaginary; therefore, the system is marginally stable and the step response oscillates indefinitely.

$\zeta = 0$;

$G4 = k_{dc} \cdot w_n^2 / (s^2 + 2 \cdot \zeta \cdot w_n \cdot s + w_n^2)$;

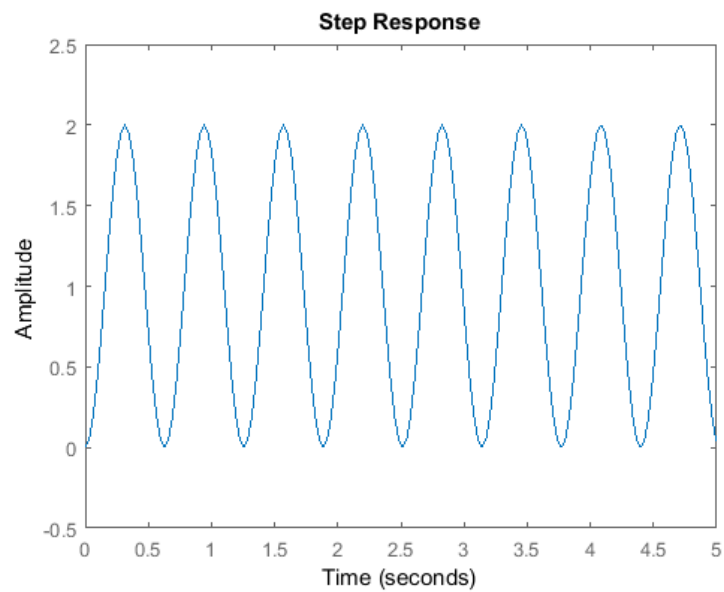
`pzmap(G4)`

`axis([-1 1 -15 15])`



`step(G4)`

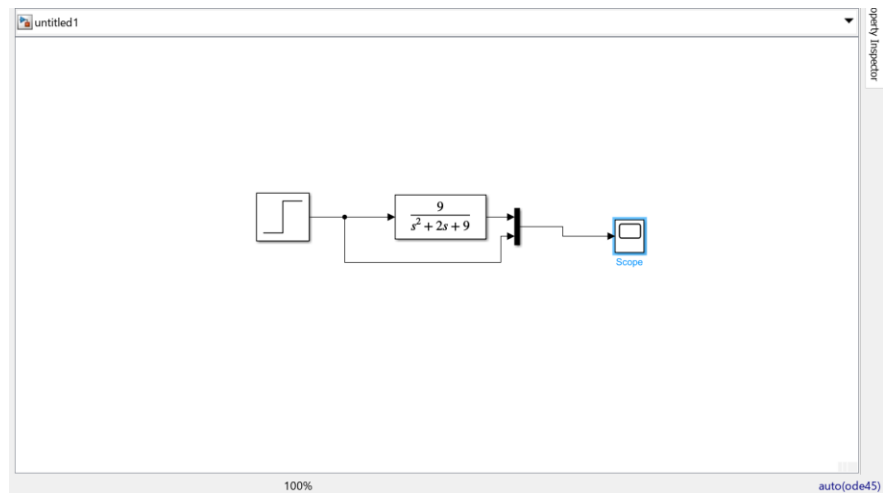
`axis([0 5 -0.5 2.5])`



More studies:

<https://in.mathworks.com/help/control/response-plots-and-data.html>

Simulink:



Output:

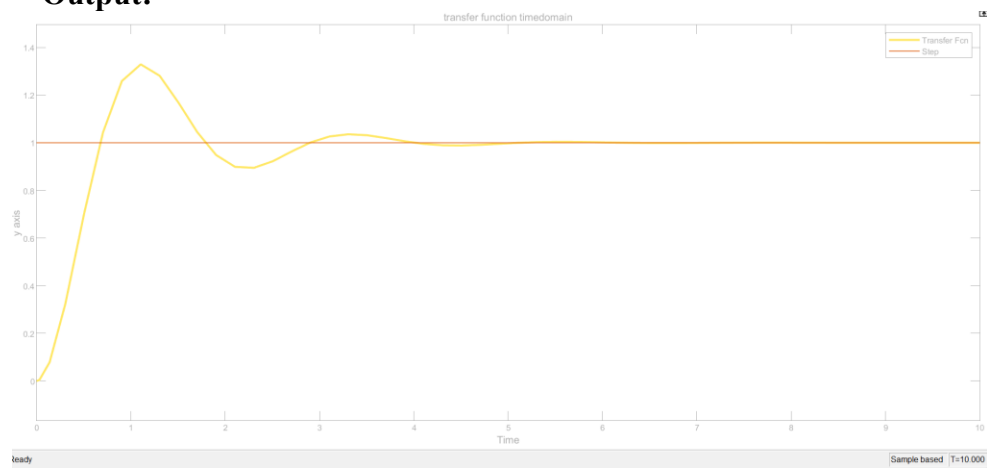


Figure 2: Step Response for the given transfer function

The above experiment can be done by varying different zeta (under damped, overdamped, critical damped and undamped).

Expt. No.:2

Date:

Estimation of steady-state error from the transfer function of a system using MATLAB

AIM/OBJECTIVE: Determine the steady state errors of a given transfer function

- **APPARATUS /COMPONENTS REQUIRED:**

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

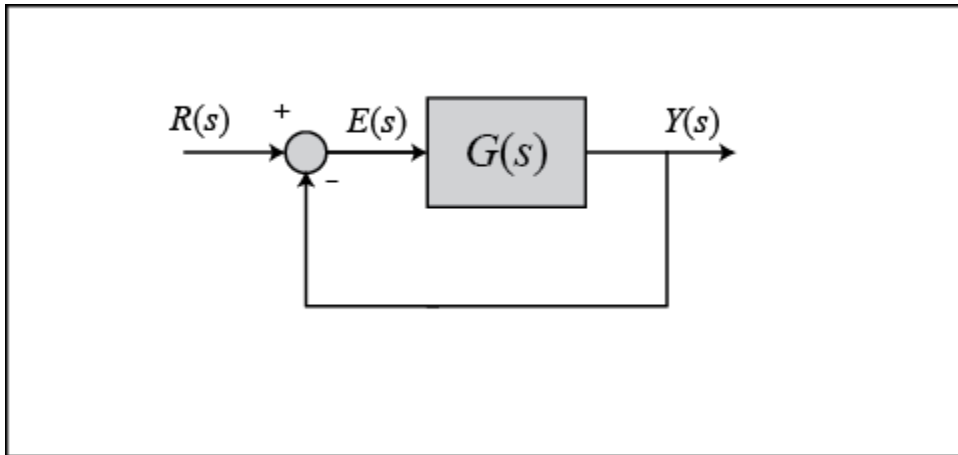
- **THEORY/METHADODOLOGY:**

Steady-state error is defined as the difference between the input (command) and the output of a system in the limit as time goes to infinity (i.e. when the response has reached steady state). The steady-state error will depend on the type of input (step, ramp, etc.) as well as the system type (0, I, or II).

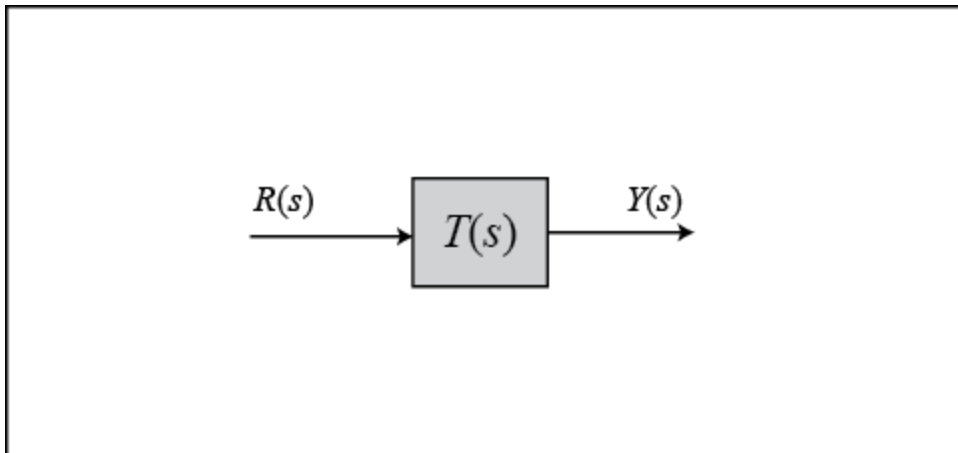
- **Note: Steady-state error analysis is only useful for stable systems. You should always check the system for stability before performing a steady-state error analysis. Many of the techniques that we present will give an answer even if the error does not reach a finite steady-state value.**

Calculating steady-state errors

Before talking about the relationships between steady-state error and system type, we will show how to calculate error regardless of system type or input. Then, we will start deriving formulas we can apply when the system has a specific structure and the input is one of our standard functions. Steady-state error can be calculated from the open- or closed-loop transfer function for unity feedback systems. For example, let's say that we have the system given below.



- This is equivalent to the following system, where $T(s)$ is the closed-loop transfer function.



We can calculate the steady-state error for this system from either the open- or closed-loop transfer function using the Final Value Theorem. **Recall that this theorem can only be applied if the subject of the limit ($sE(s)$ in this case) has poles with negative real part.**

$$e(\infty) = \lim_{s \rightarrow 0} sE(s) = \lim_{s \rightarrow 0} \frac{sR(s)}{1 + G(s)}$$

$$e(\infty) = \lim_{s \rightarrow 0} sE(s) = \lim_{s \rightarrow 0} sR(s)[1 - T(s)]$$

MATLAB Command:

```
clc
```

```
clear all
```

```
close all
```

```
h=tf([4],[1 3 4]);
```

```
a=step(h);
```

```
[y,t]=step(h);
```

```
i=length(y)
```

```
z=y(i)
```

```
c=y(end)
```

```
sserror=abs(1-c)
```

Result:

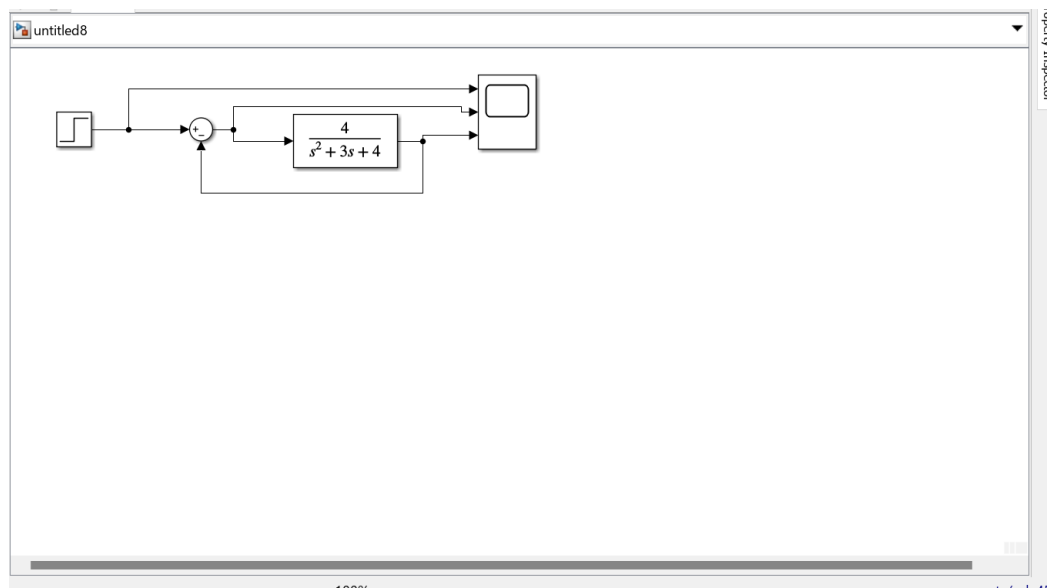
i =139

z = 0.9999

c = 0.9999

sserror =1.1622e-04

simulation:



Expt. No.:3

Date:

Stability Analysis of a vehicle suspension system by obtaining Bode, Root-locus, Nyquist plots using MATLAB

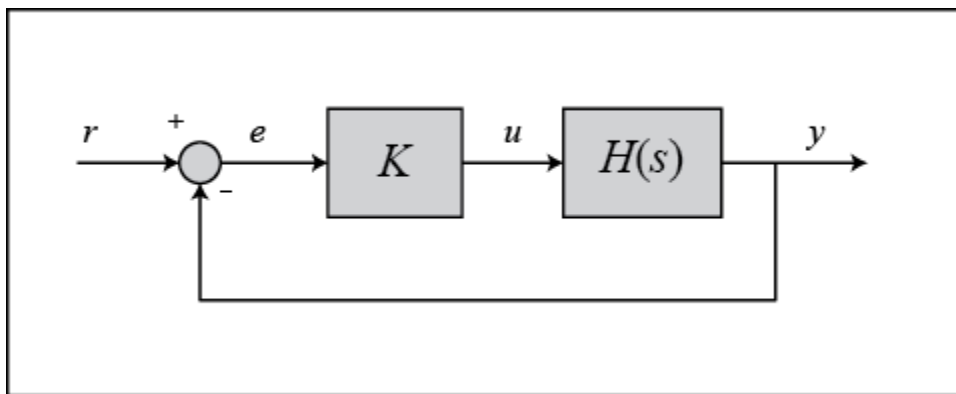
AIM/OBJECTIVE: To analyze frequency response of a system by plotting Root locus, Bode plot and Nyquist plot using MATLAB software.

• **APPARATUS /COMPONENTS REQUIRED:**

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

Theory: Stability analysis using Rootlocus

The root locus of an (open-loop) transfer function $H(s)$ is a plot of the locations (locus) of all possible closed-loop poles with some parameter, often a proportional gain K , varied between 0 and ∞ . The figure below shows a unity-feedback architecture, but the procedure is identical for any open-loop transfer function $H(s)$, even if some elements of the open-loop transfer function are in the feedback path.



The closed-loop transfer function in this case is:

$$\frac{Y(s)}{R(s)} = \frac{KH(s)}{1 + KH(s)}$$

and thus the poles of the closed-loop system are values of s such that $1 + KH(s) = 0$.

If we write $H(s) = b(s)/a(s)$, then this equation can be rewritten as:

$$(\Rightarrow a(s) + Kb(s) = 0$$

$$(3) \Rightarrow \frac{a(s)}{K} + b(s) = 0$$

Let n be the order of $a(s)$ and m be the order of $b(s)$ (the order of the polynomial corresponds to the highest power of s).

We will consider all positive values of K . In the limit as $K \rightarrow 0$, the poles of the closed-loop system are solutions of $a(s) = 0$ (poles of $H(s)$). In the limit as $K \rightarrow \infty$, the poles of the closed-loop system are solutions of $b(s) = 0$ (zeros of $H(s)$).

No matter our choice of K , **the closed-loop system has n poles**, where n is the number of poles of the open-loop transfer function $H(s)$. **The root locus then has n branches**, each branch starts at a pole of $H(s)$ and approaches a zero of $H(s)$. If $H(s)$ has more poles than zeros (as is often the case), $m < n$ and we say that $H(s)$ has **zeros at infinity**. In this case, the limit of $H(s)$ as $s \rightarrow \infty$ is zero. The number of zeros at infinity is $n - m$, the number of open-loop poles minus the number of open-loop zeros, and is the number of branches of the root locus that go to "infinity" (asymptotes).

Since the root locus consists of the locations of all possible closed-loop poles, the root locus helps us choose the value of the gain K to achieve the type of performance we desire. If any of the selected poles are on the right-half complex plane, the closed-loop system will be unstable. The poles that are closest to the imaginary axis have the greatest influence on the closed-loop response, so even if a system has three or four poles, it may still behave similar to a second- or a first-order system, depending on the location(s) of the dominant pole(s).

Root Locus Fundamentals

Poles and Zeros

- **Poles:** The values of s (complex frequency) for which the system's transfer function $G(s)H(s)$ (open-loop transfer function) becomes infinite. They are the roots of the characteristic equation $1+G(s)H(s)=0$.
- **Zeros:** The values of s for which the system's transfer function $G(s)H(s)$ becomes zero.

Characteristics of Root Locus

1. **Symmetry:** Root loci are symmetrical about the real axis.
2. **Starting and Ending Points:** Root loci start at the poles of the open-loop transfer function $G(s)H(s)$ and end at the zeros.
3. **Real Axis Segments:** The root locus exists on real-axis segments where the number of poles and zeros to the right is odd.
4. **Asymptotes:** As gain increases, the loci tend to asymptotes, which are straight lines emanating from a point known as the centroid. The angle of the asymptotes depends on the number of poles (P) and zeros (Z): $\theta_k = (2k+1)180^\circ$ where k is an integer.

Breakaway and Break-in Points

- **Breakaway Points:** Points on the real axis where multiple root loci diverge.
- **Break-in Points:** Points on the real axis where multiple root loci converge.

Stability Criterion

For a system to be stable, all closed-loop poles must lie in the left half of the s-plane (i.e., have negative real parts). The root locus plot allows for the visualization of these pole movements as the gain K varies:

- If all the loci remain in the left half-plane for a range of K, the system is stable for those values of K.
- If any loci cross into the right half-plane, the system becomes unstable for those values of K.

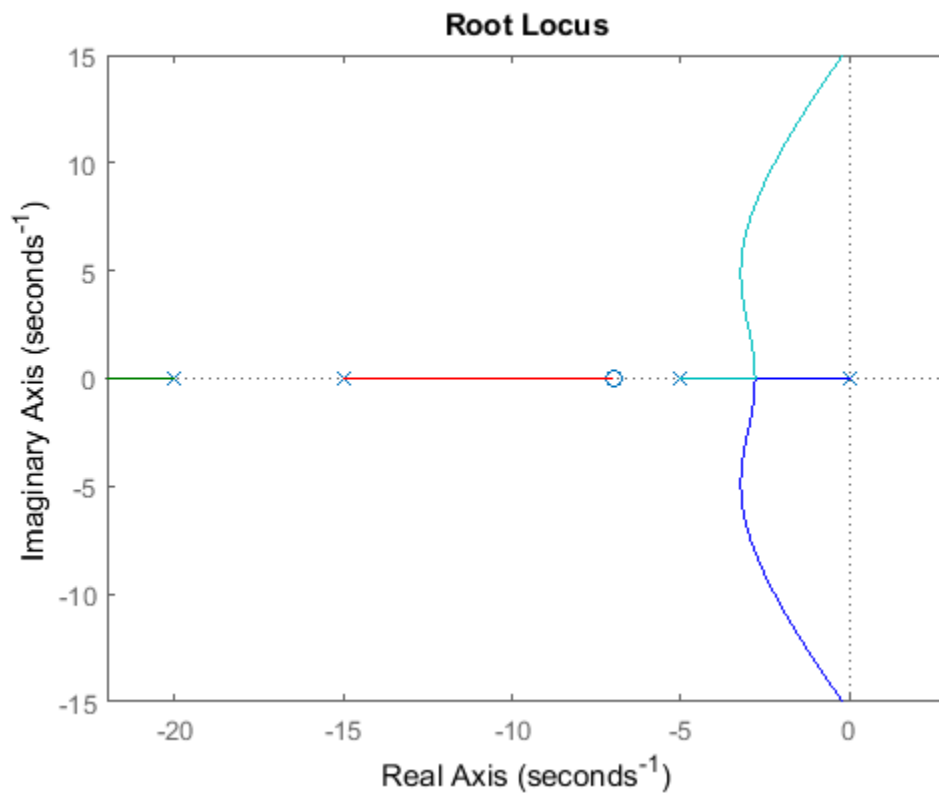
Plotting the Root Locus of a Transfer Function

Consider an open-loop system which has a transfer function of

$$(4) \quad H(s) = \frac{Y(s)}{U(s)} = \frac{s + 7}{s(s + 5)(s + 15)(s + 20)}$$

How do we design a feedback controller for the system using the root-locus method? Let's assume our design criteria are 5% overshoot and 1 second rise time. Create an [m-file](#) titled rl.m. Within this file, create the transfer function model and employ the rlocus command as follows:

```
s = tf('s');  
sys = (s + 7)/(s*(s + 5)*(s + 15)*(s + 20));  
rlocus(sys)  
axis([-22 3 -15 15])
```



Bode plots are a fundamental tool in control system engineering, used to analyze the frequency response of linear, time-invariant systems. They provide a graphical representation of a system's

gain and phase shift across a range of frequencies. Bode plots are particularly useful for assessing system stability and performance in the frequency domain.

Components of a Bode Plot

A Bode plot consists of two separate graphs:

1. **Magnitude Plot:** Shows how the system's gain (magnitude of the output/input ratio) varies with frequency, usually expressed in decibels (dB).
2. **Phase Plot:** Shows how the phase angle of the system's output relative to its input varies with frequency, usually expressed in degrees.

Key Concepts

- **Gain Margin (GM):** The amount of gain increase required to bring the system to the verge of instability. It is measured at the phase crossover frequency, where the phase angle is -180 degrees.
- **Phase Margin (PM):** The additional phase lag required to bring the system to the verge of instability. It is measured at the gain crossover frequency, where the gain is 0 dB.

Frequency Points:

- **Gain Crossover Frequency (ω_{gc}):** The frequency at which the gain is 0 dB.
- **Phase Crossover Frequency (ω_{pc}):** The frequency at which the phase angle is -180 degrees.

Stability Criteria

1. **Gain Margin:**
 - A positive gain margin indicates that the system can tolerate some increase in gain before becoming unstable.
 - Gain margin $GM = 20 \log_{10}(1/|G(j\omega_{pc})|)$, where $|G(j\omega_{pc})|$ is the magnitude at phase crossover frequency.
2. **Phase Margin:**

- A positive phase margin indicates that the system can tolerate some additional phase lag before becoming unstable.
- Phase margin $PM = 180^\circ + \angle G(j\omega_{gc})$ where $\angle G(j\omega_{gc})$ is the phase angle at gain crossover frequency.

1. Analyze Stability:

- If both gain margin and phase margin are positive, the system is stable.
- If either margin is zero or negative, the system is unstable.

Plot bode plot of given transfer function and find gain and phase margins using MATLAB.

```
% Transfer Function = 50/(s^3+9s^2+30s+40)

clear all

clc

n = [50];

d = [1 9 30 40];

sys = tf(n,d)

w = logspace(-1,2,4)

bode(sys)

grid;

[a,phase] = bode(sys,w);

mag = 20*log10(a);

x = w';

[G_Margin,P_Margin,w_phase_crossover,w_gain_crossover] = margin(sys)

Gain_Margin_dB = 20*log10(G_Margin)

disp('Frequency(rad/sec) Magnitude(dB) Phase(degree)')
```

```

for i = 1:4

fprintf('%f\t\t%f\t\t%f\n',x(i),mag(i),phase(i))

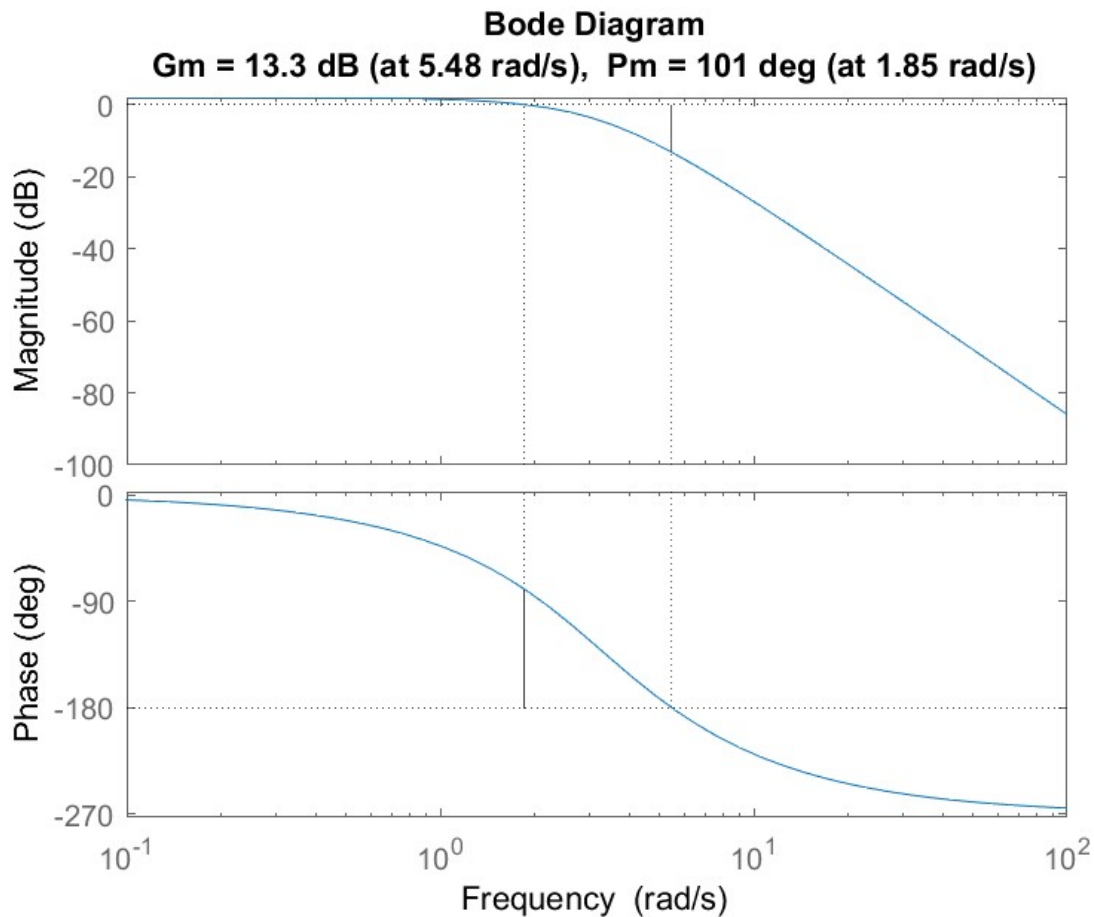
end

```

Results

Transfer function: $\frac{50}{s^3 + 9s^2 + 30s + 40}$

GM = 4.6019, PM = 100.6674, Wcg = 5.4782, Wcp = 1.8483



Stability analysis using Nyquist Plot:

Nyquist plots are a powerful tool in control system engineering, used to analyze the stability of linear, time-invariant systems. The Nyquist plot is a graphical representation of the frequency

response of a system's open-loop transfer function $G(s)H(s)$. It provides insight into the stability of the closed-loop system by examining how the open-loop frequency response encircles the critical point $(-1,0)$ in the complex plane.

Frequency Response

The Nyquist plot represents the frequency response of a system over a range of frequencies, usually from $-\infty$ to ∞ . The plot is created by mapping the transfer function $G(s)H(s)$ onto the complex plane as $s=j\omega$ varies, where ω is the frequency.

Nyquist Criterion

The Nyquist stability criterion relates the open-loop frequency response to the closed-loop stability of the system. It is based on the principle of mapping and the argument principle in complex analysis.

Key Concepts:

- **Encirclement:** The number of times the Nyquist plot encircles the critical point $(-1,0)$ in the complex plane.
- **Poles of $G(s)H(s)$:** The number of open-loop poles of the transfer function $G(s)H(s)$ that lie in the right half of the s -plane.

Nyquist Criterion:

- For a system to be stable, the number of clockwise encirclements of the critical point $(-1,0)$ by the Nyquist plot must equal the number of poles of $G(s)H(s)$ in the right half-plane.
- If there are no open-loop poles in the right half-plane, the system is stable if the Nyquist plot does not encircle the critical point $(-1,0)$.

Step-by-Step Procedure for Nyquist Plot Analysis

1. **Determine the Open-Loop Transfer Function:** $G(s)H(s)=N(s)/D(s)$ where $N(s)$ and $D(s)$ are the numerator and denominator polynomials, respectively.
2. **Construct the Nyquist Plot:**
 - Map the frequency response of $G(j\omega)H(j\omega)$ over a range of frequencies from $-\infty$ to ∞ .
 - Include the positive and negative frequency ranges to ensure a complete plot.
3. **Analyze Encirclements:**
 - Count the number of clockwise encirclements of the critical point $(-1,0)$.
 - Determine the number of poles of $G(s)H(s)$ in the right half-plane.
4. **Apply the Nyquist Criterion:**
 - Use the Nyquist criterion to assess the stability of the closed-loop system based on the encirclements and the number of right-half-plane poles.

% Define the numerator and denominator of the transfer function

num = [1]; % Example numerator coefficients

den = [1 3 2]; % Example denominator coefficients

% Create the transfer function

sys = tf(num, den);

% Plot the Nyquist diagram

figure;

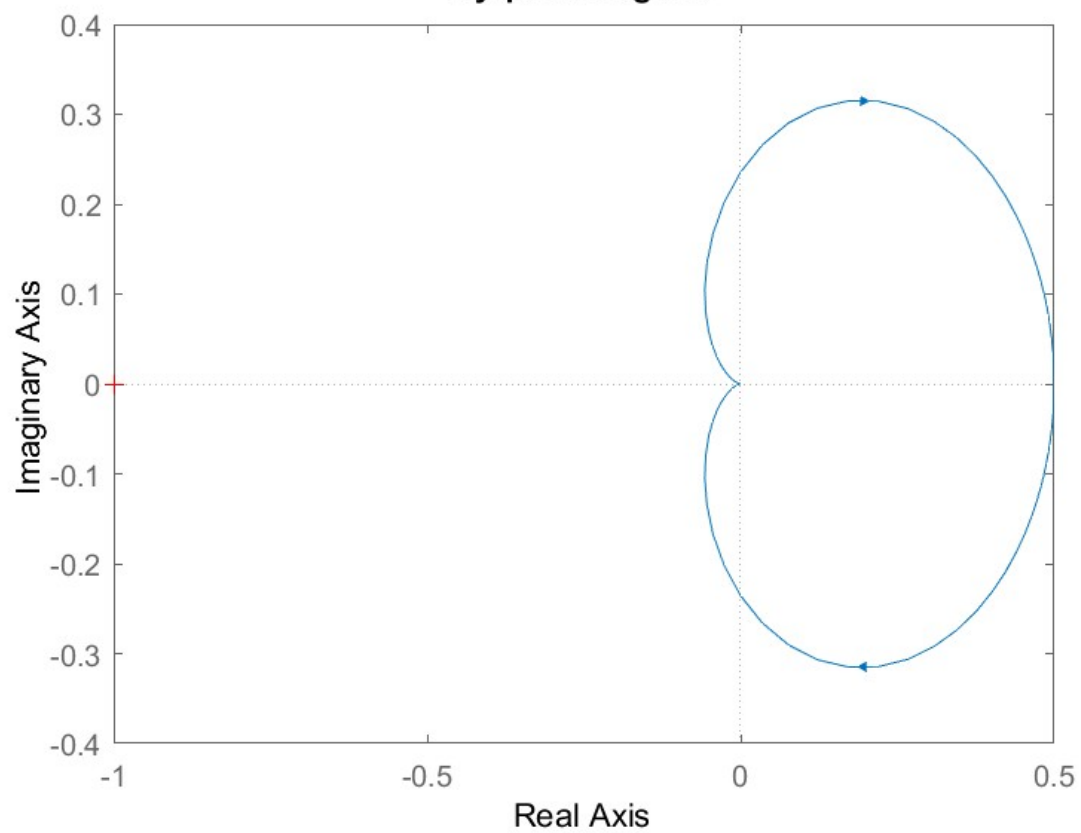
nyquist(sys);

grid on;

title('Nyquist Plot of the System');

% Analyze the stability using the Nyquist plot % The plot and the number of encirclements around $(-1,0)$ provide the stability information

Nyquist Diagram



Expt. No.:4

Date:

Stability analysis of first order system and second order system

AIM/OBJECTIVE: To perform Stability analysis of first order system and second order system.

• **APPARATUS /COMPONENTS REQUIRED:**

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

Theory:

A first-order system is characterized by a single energy storage element and a transfer function with one pole. The stability of a first-order system is determined by the location of its pole in the complex plane. If the pole is in the left half-plane, the system is stable, meaning it will return to equilibrium after a disturbance. Conversely, if the pole is in the right half-plane, the system is unstable and will diverge over time.

A second-order system is characterized by two energy storage elements and a transfer function with two poles. The stability of a second-order system depends on the damping ratio and natural frequency, which determine the poles' locations in the complex plane. For stability, the real parts of both poles must be negative. The damping ratio also classifies the system's response: overdamped, critically damped, underdamped, or undamped, affecting how quickly the system returns to equilibrium and whether it oscillates during this process.

MATLAB Code:

% Stability analysis of **first order system**

```

clc;
clear all;
close all;
% Define the sample time
T = 1;
% Define the numerator and denominator of the discrete transfer function
num = [0.05];
den = [1 -0.95];
% Create the discrete transfer function with the sample time
G = tf(num, den, T);
% Define the gain
K = 10;
% Close the feedback loop with unity feedback
sys = feedback(K * G, 1);
% Plot the step response
figure;
step(sys);
title('Step Response of Discrete System with Unity Feedback');
data = stepinfo(sys); % Get time domain specifications of the given system
% Display the legend
legend('Step Response of Discrete System with Unity Feedback');

% Display the time-domain specifications

disp('Time Domain Specifications:');

disp(data);

% Stability analysis: check the poles of the closed-loop system

poles = pole(sys);

disp('Poles of the closed-loop system:');

disp(poles);

% Check stability

if all(abs(poles) < 1)

    disp('The system is stable.');
```

else

```

    disp('The system is unstable.');
```

end

Stability analysis of second order system

% Define the numerator and denominator of the transfer function

num = [420 0]; % Coefficients of the numerator (420s)

den = [1 620 4000]; % Coefficients of the denominator ($s^2 + 620s + 4000$)

% Create the transfer function

sys = tf(num, den);

% Display the transfer function

disp('Transfer Function:');

disp(sys);

% Check the stability using the poles of the system

poles = pole(sys);

disp('Poles of the system:');

disp(poles);

% Determine if the system is stable

if all(real(poles) < 0)

 disp('The system is stable.');

elseif any(real(poles) == 0)

 disp('The system is marginally stable.');

else

 disp('The system is unstable.');

end

% Plot the step response to visualize the system behavior

figure;

step(sys);

title('Step Response');

grid on;

% Plot the impulse response to visualize the system behavior

figure;

impz(sys);

title('Impulse Response');

grid on;

Expt. No.:5

Date:

STABILITY ANALYSIS USING ROUTH- HURWITZ METHOD

AIM/OBJECTIVE: To determine the stability of a system using Routh Hurwitz method.

- APPARATUS /COMPONENTS REQUIRED:**

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

Theory:

The theory of network synthesis states that any pole of the system lies on the right hand side of the origin of the s plane, it makes the system unstable. On the basis of this condition A. Hurwitz and E.J.Routh started investigating the necessary and sufficient conditions of stability of a system. We will discuss two criteria for stability of the system. A first criterion is given by A. Hurwitz and this criterion is also known as Hurwitz Criterion for stability or Routh Hurwitz Stability Criterion. With the help of characteristic equation, we will make a number of Hurwitz determinants in order to find out the stability of the system. We define characteristic equation of the system as:

$$a_0 s^n + a_1 s^{n-1} + a_2 s^{n-2} + \dots + a_{n-1} s^1 + a_n$$

where n determinants for nth order characteristic equation

Steps:

Enter the program in the editor window.

Execute the program.

Enter the coefficients of characteristic equation in the command window.

Finally save the program and run it.

The Routh matrix is obtained in the command window.

Determine Stability:

- Examine the first column of the Routh array. The number of sign changes in this column indicates the number of roots with positive real parts (i.e., the number of poles in the right half-plane).
- If there are no sign changes, all roots are in the left half-plane, and the system is stable.
- If there are sign changes, the system is unstable, and the number of sign changes corresponds to the number of unstable poles.

Program:

```
clear
clc
%% firstly it is required to get first two row of routh matrix
e=input('enter the coefficients of characteristic equation: '); disp('--')
l=length(e);
m=mod(l,2);
if m==0
    a=rand(1,(l/2));
    b=rand(1,(l/2));
    for i=1:(l/2)
        a(i)=e((2*i)-1); b(i)=e(2*i);
    end
else e1=[e 0];
    a=rand(1,((l+1)/2));
    b=[rand(1,((l-1)/2)),0];
    for i=1:((l+1)/2)
        a(i)=e1((2*i)-1); b(i)=e1(2*i);
    end
end
%% now we generate the remaining rows of routh
matrix l1=length(a);
c=zeros(l,l1);
c(1,:)=a;
c(2,:)=b;
for m=3:l
    for n=1:l1-1
        c(m,n)=-(1/c(m-1,1))*det([c((m-2),1) c((m-2),(n+1));c((m-1),1) c((m-1),(n+1))]);
    end
end
disp('the routh matrix:') disp(c)
%% now we check the stability of system
if c(:,1)>0
    disp('System is Stable')
else
    disp('System is Unstable');
end
```

OUTPUT:

enter the coefficients of characteristic equation: [1 1 3 1 6] -

the routh matrix:

1 3 6

1 1 0

2 6 0

-2 0 0

6 0 0

System is Unstable

Analysis of a Second Order Translational Mechanical System with Different Controller Configurations

Expt. No.:6

Date:

Analysis of a Second Order Translational Mechanical System with Different Controller Configurations using MATLAB

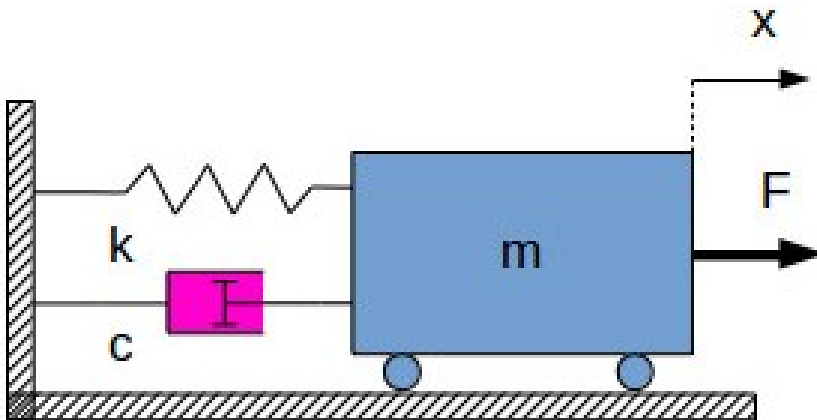
AIM/OBJECTIVE:

1. Derive the open-loop and closed-loop transfer function model of the given system.
2. Develop the block diagrams of the closed-loop system including various controller configurations (P, PI, PD, PID) and plot the step response in each of the above cases by varying K_p , K_i and K_d values and find the optimum values to achieve the desired response in each case.

• APPARATUS /COMPONENTS REQUIRED:

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

Data Given:



$M = 1 \text{ kg}$, $c = 10 \text{ N.s/m}$, $k = 1 \text{ N/m}$, $F(s) = 1 \text{ N}$

Theory:

The modeling equation of this system is

$$M\ddot{x} + b\dot{x} + kx = F \quad (1)$$

Taking the Laplace transform of the modeling equation (1), we get

$$Ms^2X(s) + bsX(s) + kX(s) = F(s)$$

The transfer function between the displacement $X(s)$ and the input $F(s)$ then becomes

$$\frac{X(s)}{F(s)} = \frac{1}{Ms^2 + bs + k}$$

Let $M = 1\text{kg}$ $b = 10\text{ N.s/m}$ $k = 20\text{ N/m}$ $F(s) = 1$

Plug these values into the above transfer function

$$\frac{X(s)}{F(s)} = \frac{1}{s^2 + 10s + 20}$$

The goal of this problem is to show you how each of K_p , K_i and K_d contributes to obtain

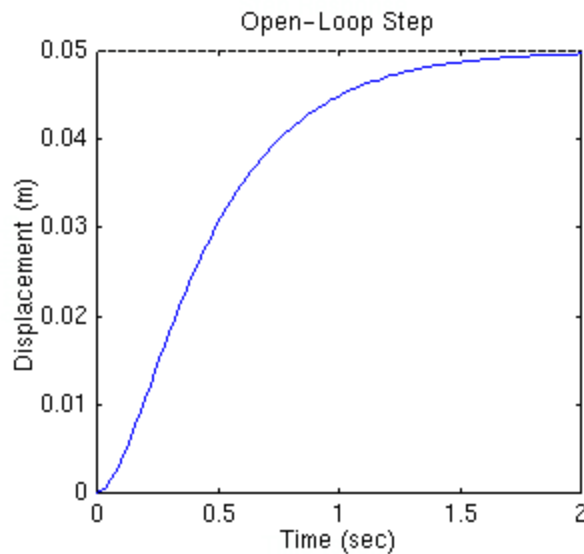
- Fast rise time
- Minimum overshoot
- No steady-state error

Open-loop step response

Let's first view the open-loop step response. Create a new [m-file](#) and add in the following code:

```
num=1;  
den=[1 10 20];  
plant=tf(num,den);  
step(plant)
```

Running this m-file in the MATLAB command window should give you the plot shown below.



The DC gain of the plant transfer function is 1/20, so 0.05 is the final value of the output to an unit step input. This corresponds to the steady-state error of 0.95, quite large indeed. Furthermore, the rise time is about one second, and the settling time is about 1.5 seconds. Let's design a controller that will reduce the rise time, reduce the settling time, and eliminates the steady-state error.

Proportional control

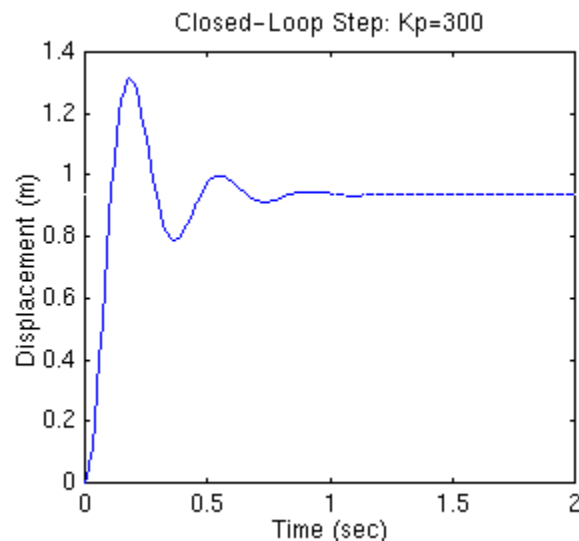
From the table shown above, we see that the proportional controller (K_p) reduces the rise time, increases the overshoot, and reduces the steady-state error. The closed-loop transfer function of the above system with a proportional controller is:

$$\frac{X(s)}{F(s)} = \frac{K_p}{s^2 + 10s + (20 + K_p)}$$

Let the proportional gain (K_p) equal 300 and change the m-file to the following:

```
Kp=300;  
contr=Kp;  
sys_cl=feedback(contr*plant,1);  
t=0:0.01:2;  
step(sys_cl,t)
```

Running this m-file in the MATLAB command window should give you the following plot.



Note:

- The MATLAB function called **feedback** was used to obtain a closed-loop transfer function directly from the open-loop transfer function (instead of computing closed-loop transfer function by hand).

- The above plot shows that the proportional controller reduced both the rise time and the steady-state error, increased the overshoot, and decreased the settling time by small amount.

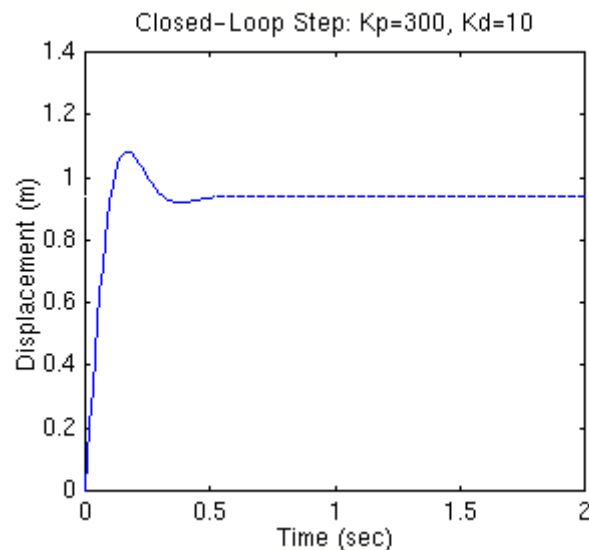
Proportional-Derivative control

Now, let's take a look at a PD control. From the table shown above, we see that the derivative controller (K_d) reduces both the overshoot and the settling time. The closed-loop transfer function of the given system with a PD controller is:

$$\frac{X(s)}{F(s)} = \frac{K_D s + K_P}{s^2 + (10 + K_D)s + (20 + K_P)}$$

Let K_p equal 300 as before and let K_d equal 10. Enter the following commands into an [m-file](#) and run it in the MATLAB command window.

```
Kp=300;
Kd=10;
contr=tf([Kd Kp],1);
sys_cl=feedback(contr*plant,1);
t=0:0.01:2;
step(sys_cl,t)
```



This plot shows that the derivative controller reduced both the overshoot and the settling time, and had a small effect on the rise time and the steady-state error.

Proportional-Integral control

Before going into a PID control, let's take a look at a PI control. From the table, we see that an integral controller (K_i) decreases the rise time, increases both the overshoot and the

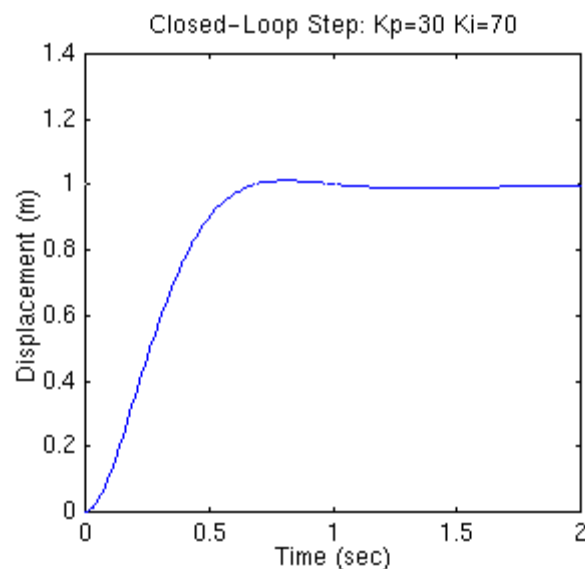
settling time, and eliminates the steady-state error. For the given system, the closed-loop transfer function with a PI control is:

$$\frac{X(s)}{F(s)} = \frac{K_P s + K_I}{s^3 + 10s^2 + (20 + K_P)s + K_I}$$

Let's reduce the K_p to 30, and K_i equal 70. Create an new [m-file](#) and enter the following code.

```
Kp=30;
Ki=70;
contr=tf([Kp Ki],[1 0]);
sys_cl=feedback(contr*plant,1);
t=0:0.01:2;
step(sys_cl,t)
```

Run this m-file in the MATLAB command window, and you should get the following plot.



We have reduced the proportional gain (K_p) because the integral controller also reduces the rise time and increases the overshoot as the proportional controller does (double effect). The above response shows that the integral controller eliminated the steady-state error.

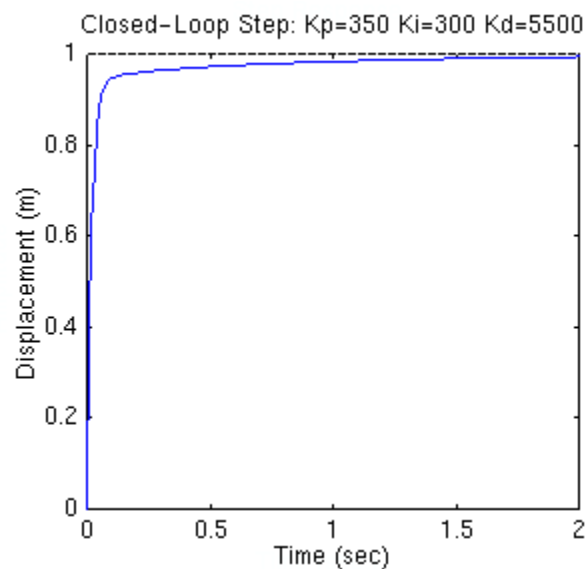
[Proportional-Integral-Derivative control](#)

Now, let's take a look at a PID controller. The closed-loop transfer function of the given system with a PID controller is:

$$\frac{X(s)}{F(s)} = \frac{K_D s^2 + K_P s + K_I}{s^3 + (10 + K_D)s^2 + (20 + K_P)s + K_I}$$

After several trial and error runs, the gains $K_p=350$, $K_i=300$, and $K_d=50$ provided the desired response. To confirm, enter the following commands to an m-file and run it in the command window. You should get the following step response.

```
Kp=350;  
Ki=300;  
Kd=50;  
contr=tf([Kd Kp Ki],[1 0]);  
sys_cl=feedback(contr*plant,1);  
t=0:0.01:2;  
step(sys_cl,t)
```



Now, we have obtained a closed-loop system with no overshoot, fast rise time, and no steady-state error.

MATLAB CODES & RESULTS

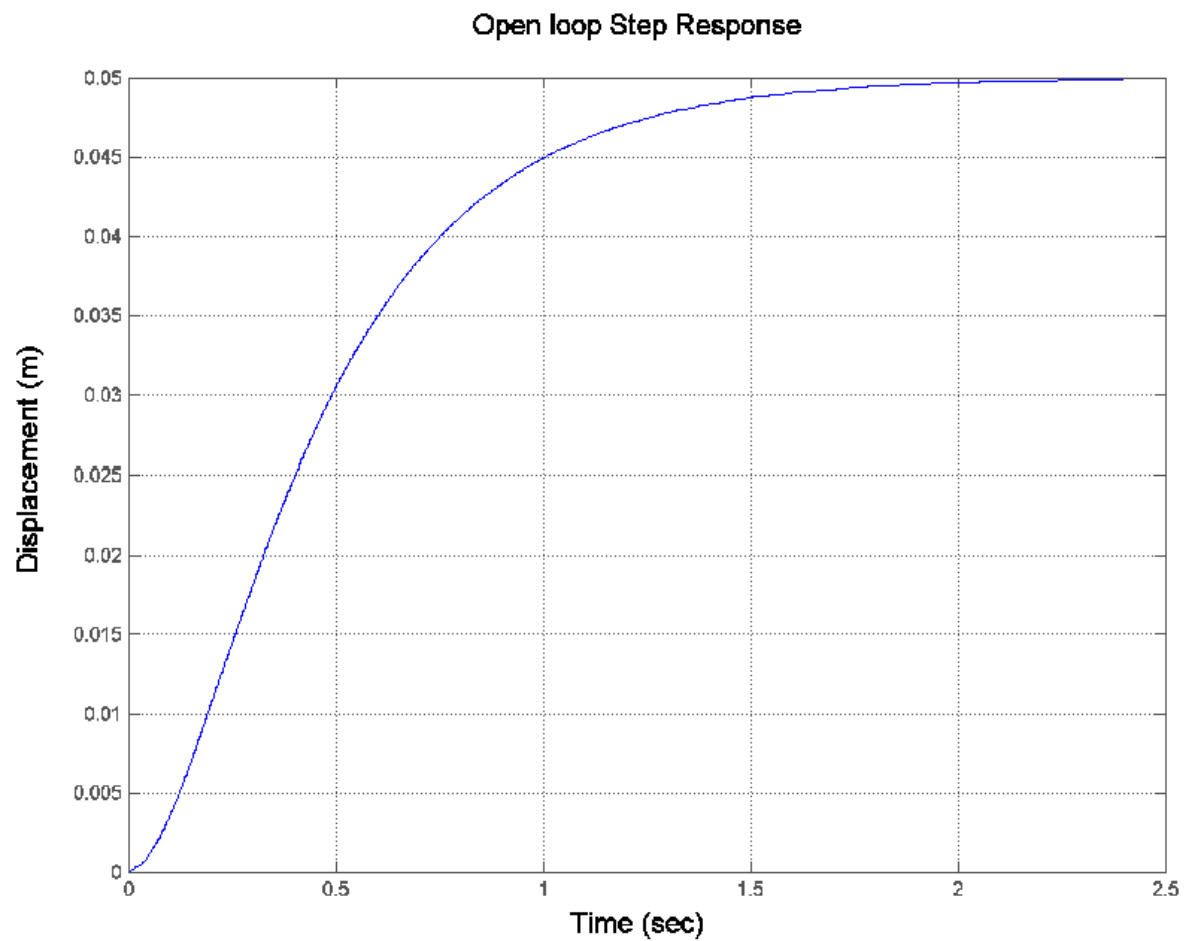
```
% Transfer function of a simple second order Control system  
% Step response of the open loop system  
clc;  
num=1;
```

```
den=[1 10 20];  
plant=tf(num,den)  
step(plant)
```

Out Put:-

Transfer function:

$$\frac{1}{s^2 + 10s + 20}$$



```

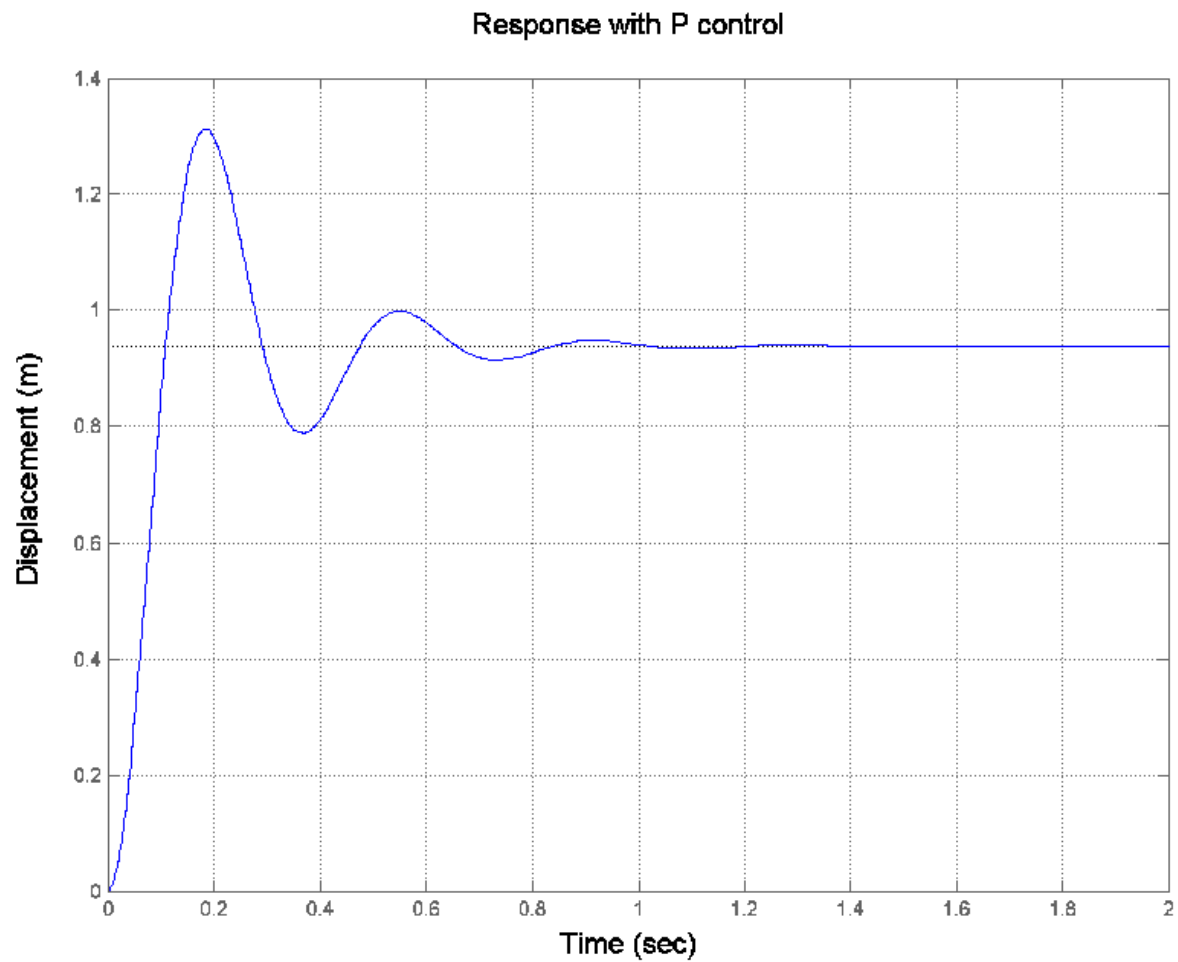
% Closed loop response
%Proportional control
clc
Kp=300;
contr=Kp;
num=1;
den=[1 10 20];
plant=tf(num,den);
sys_cl=feedback(contr*plant,1)
t=0:0.01:2;
step(sys_cl,t)

```

Out Put:-

Closed loop Transfer function:

$$\frac{300}{s^2 + 10 s + 320}$$



%Proportional plus Integral control

clc

Kp=30;

Ki=70;

num=1;

den=[1 10 20];

plant=tf(num,den);

contr=tf([Kp Ki],[1 0]);

sys_cl=feedback(contr*plant,1)

t=0:0.01:2;

```
step(sys_cl,t)
```

Out Put:-

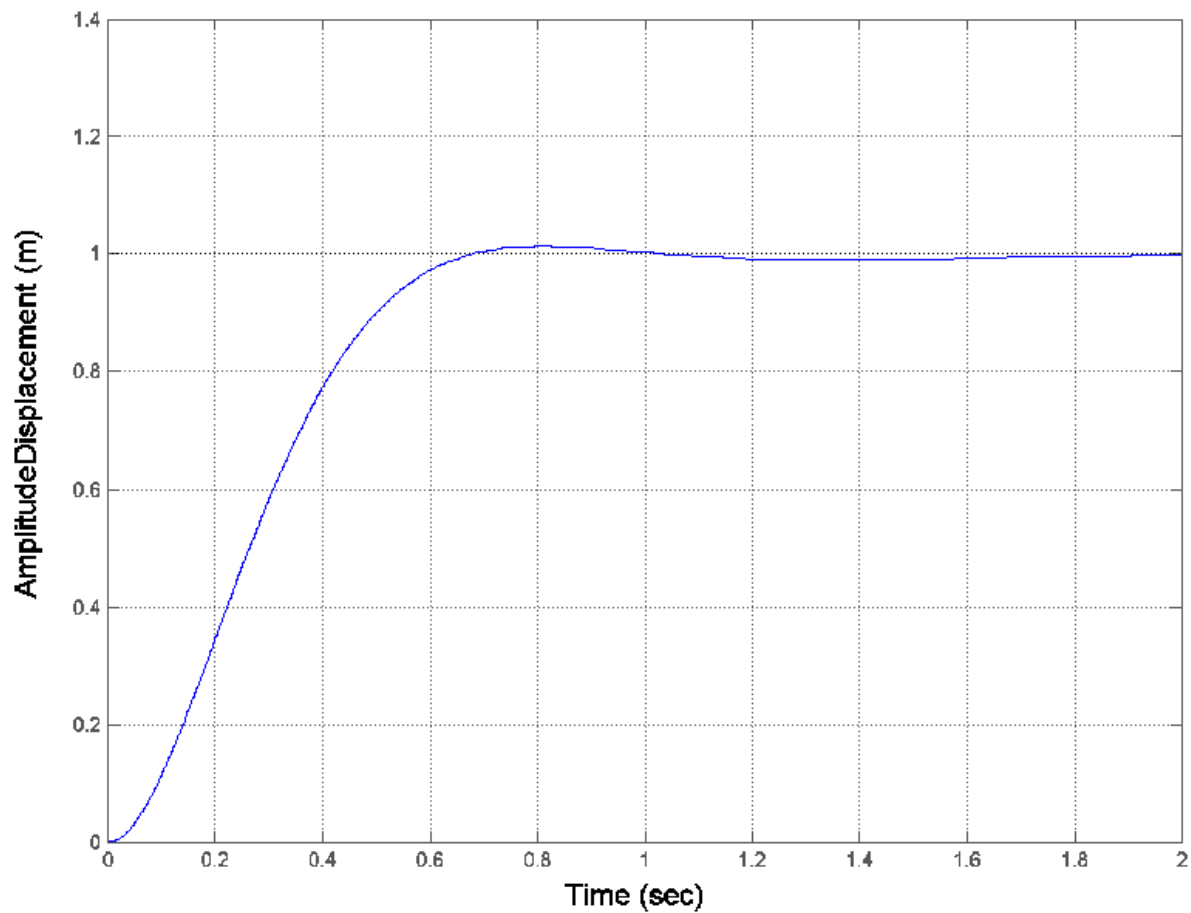
Controller Transfer function =

$$\frac{30s + 70}{s}$$

Overall system Transfer function=

$$\frac{30s + 70}{s^3 + 10s^2 + 50s + 70}$$

Pi controller response



%Proportional plus Derivative control

```
clc
```

```
num=1;
```

```
den=[1 10 20];
```

```
plant=tf(num,den);
```

```

Kp=300;
Kd=10;
contr=tf([Kd Kp],1)
sys_cl=feedback(contr*plant,1)
t=0:0.01:2;
step(sys_cl,t)

```

Out Put:-

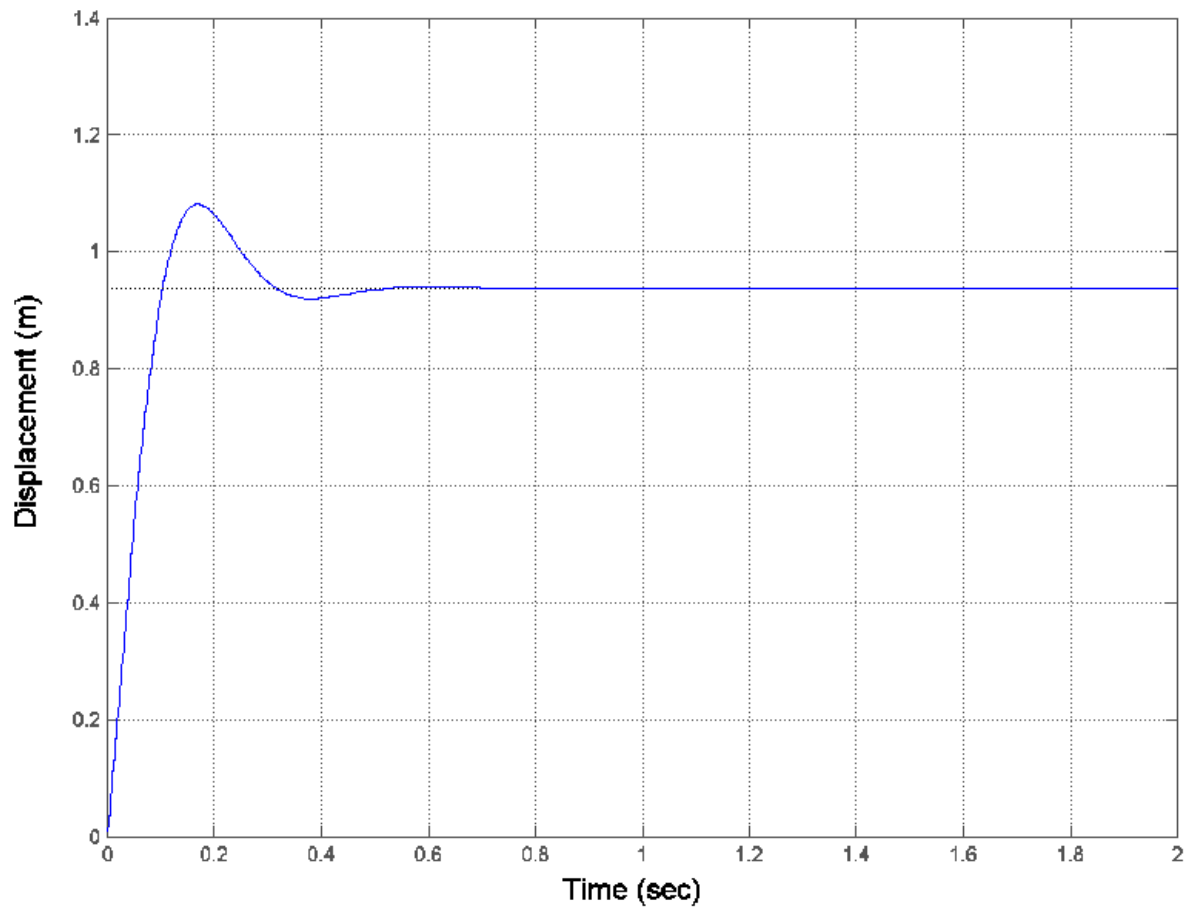
Controller Transfer function =

$$10 s + 300$$

Overall system Transfer function=

$$\frac{10 s + 300}{s^2 + 20 s + 320}$$

Response with PD controller



%Proportional plus Integral plus Derivative control

clc

num=1;

den=[1 10 20];

plant=tf(num,den);

Kp=350;

Ki=300;

Kd=50;

contr=tf([Kd Kp Ki],[1 0]);

sys_cl=feedback(contr*plant,1);

t=0:0.01:2;

step(sys_cl,t)

Out Put:-

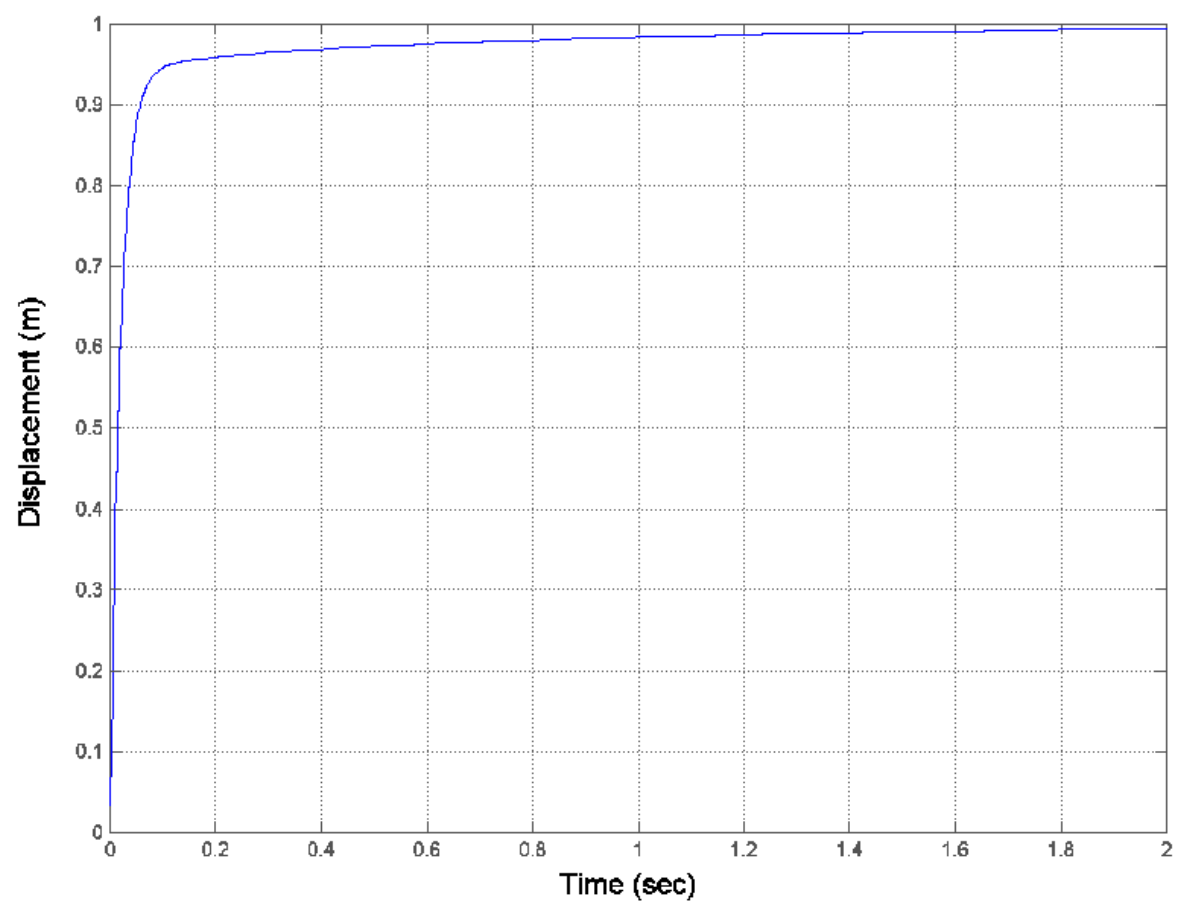
Controller Transfer function:

$$\frac{50 s^2 + 350 s + 300}{s}$$

Overall system Transfer function:

$$\frac{50 s^2 + 350 s + 300}{s^3 + 60 s^2 + 370 s + 300}$$

PID controller Response



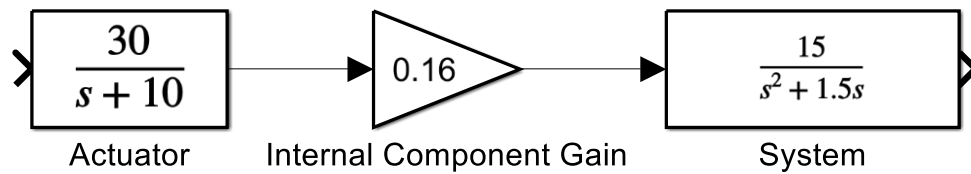
Expt. No.:7

Date:

Response Analysis of Open-loop and Closed-loop Systems Under Different Operating Conditions (with and without disturbance and controller) using Simulink

AIM/OBJECTIVE:

Perform a comprehensive analysis on the following system under different operating conditions and comment on your observations on its response with respect to desired output characteristics.

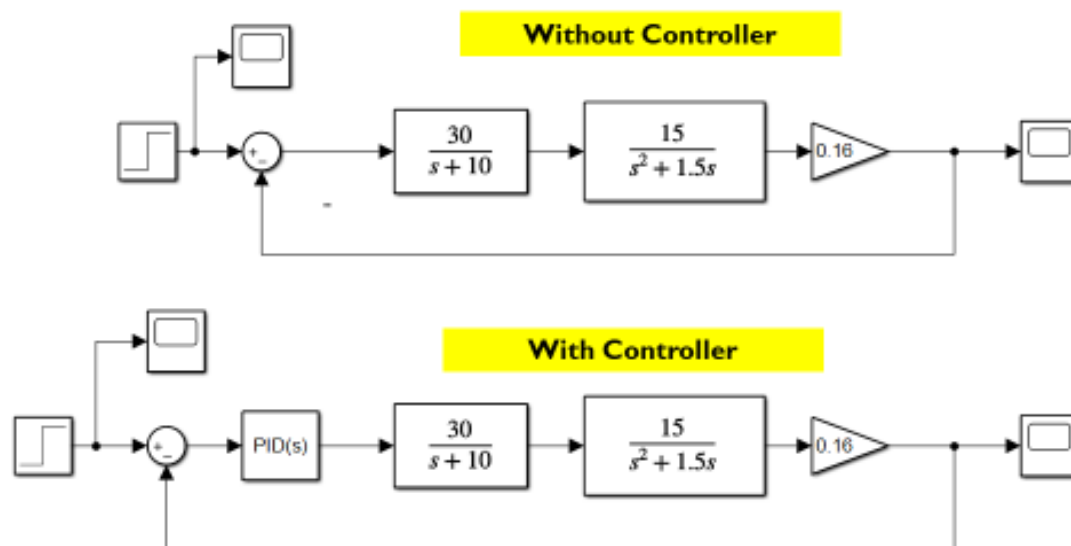
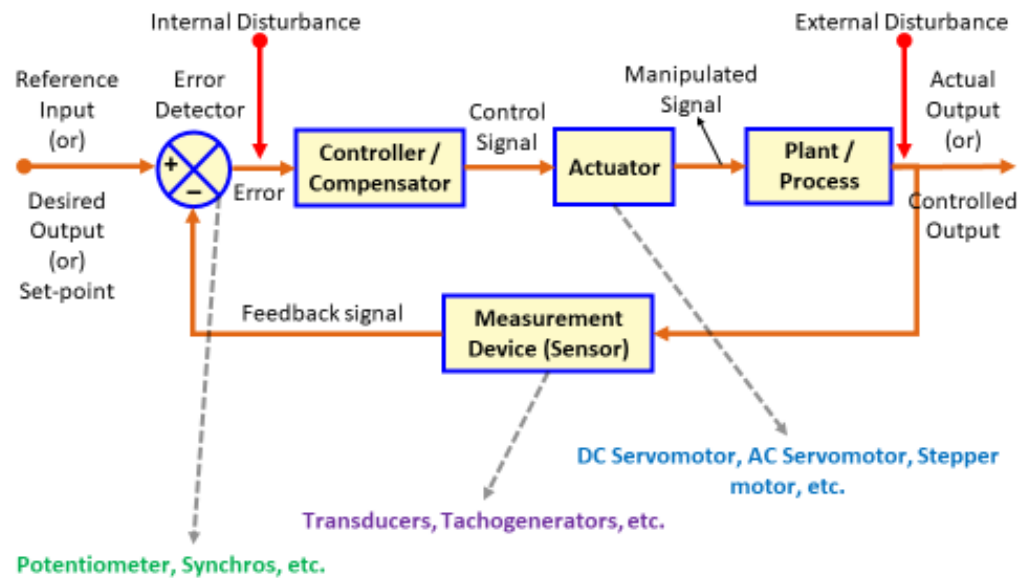


- Open-loop response without controller and without disturbance
- Closed-loop response without controller and without disturbance
- Open-loop response with controller and without disturbance
- Closed-loop response with controller and without disturbance
- Open-loop response without controller and with disturbance (both internal & external individually)
- Closed-loop response without controller and with disturbance (both internal & external individually)
- Open-loop response with controller and with disturbance (both internal & external individually)
- Closed-loop response with controller and with disturbance (both internal & external individually)

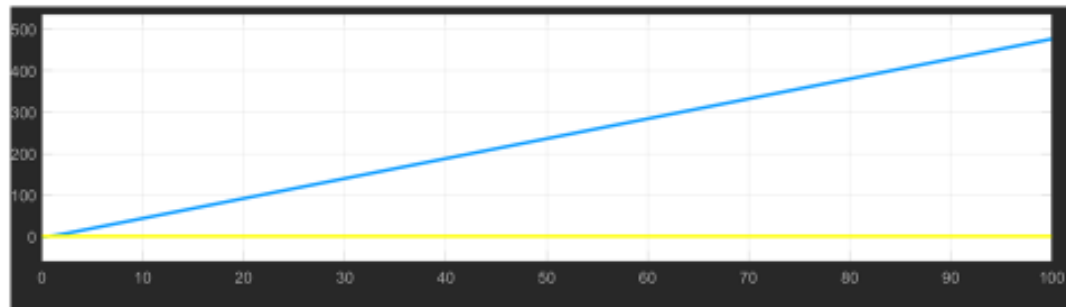
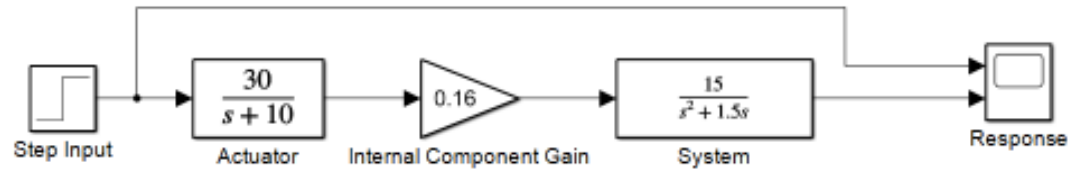
APPARATUS /COMPONENTS REQUIRED:

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

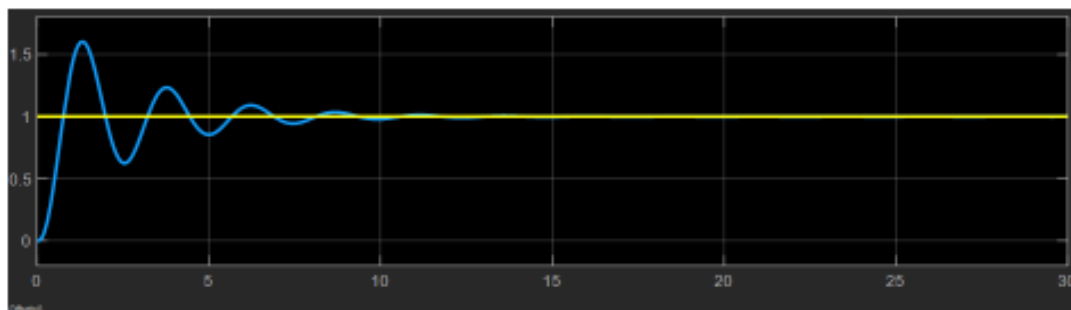
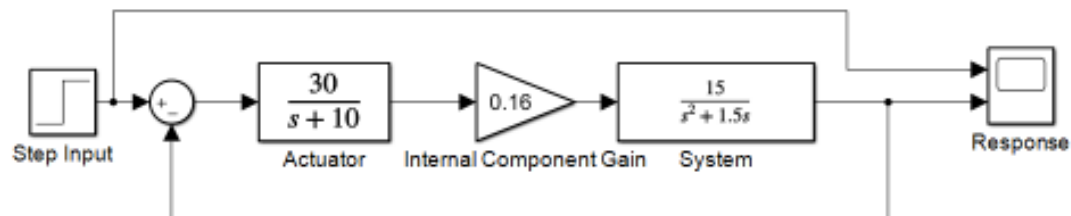
Theory:



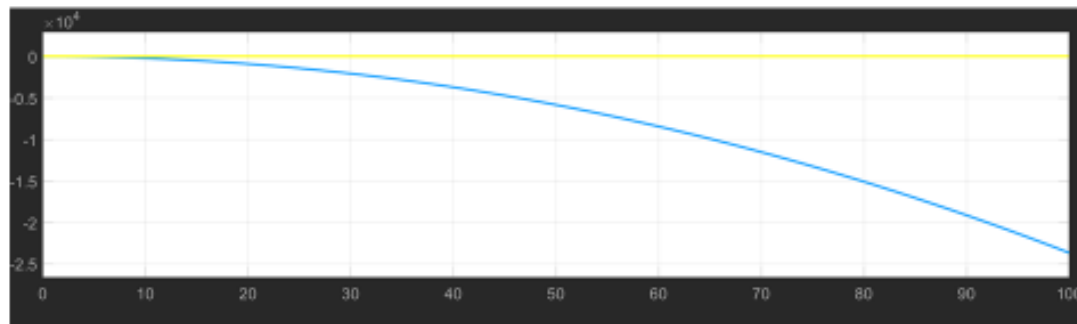
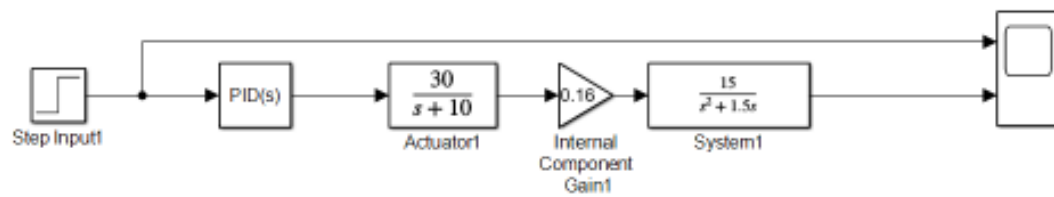
1. Open-loop response without controller and without disturbance



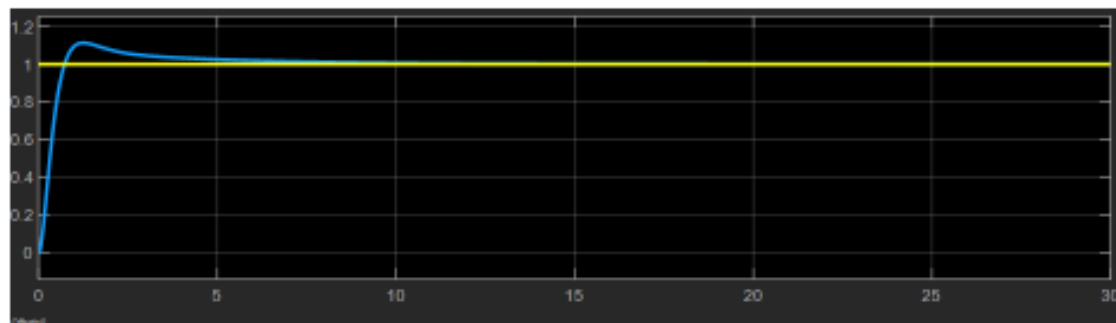
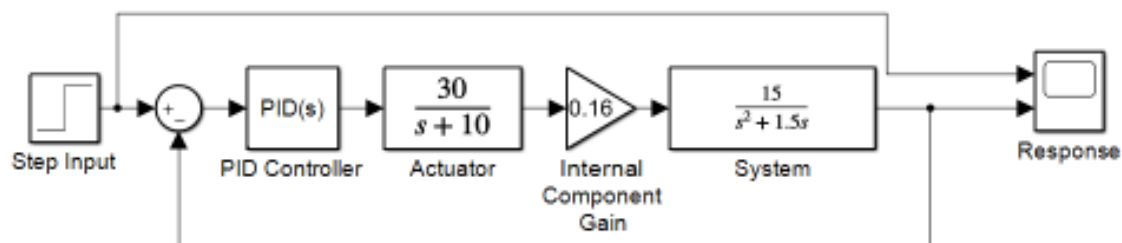
2. Closed-loop response without controller and without disturbance



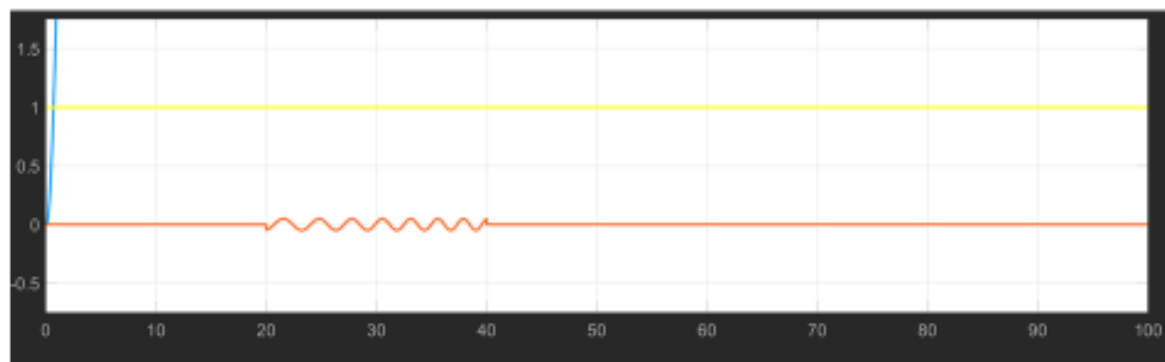
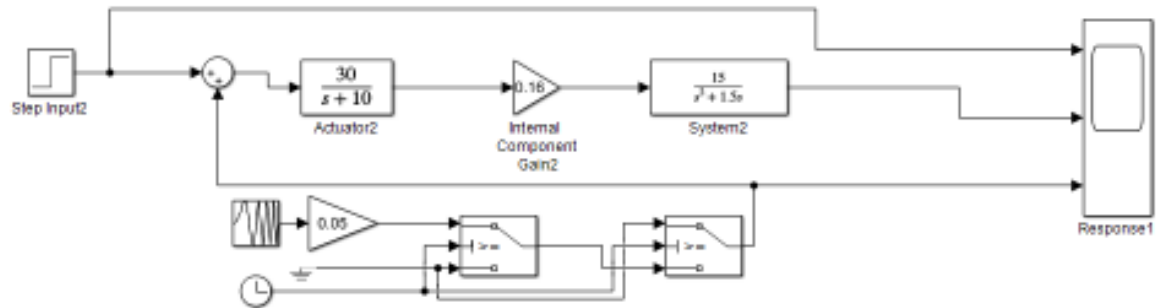
3. Open-loop response with controller and without disturbance



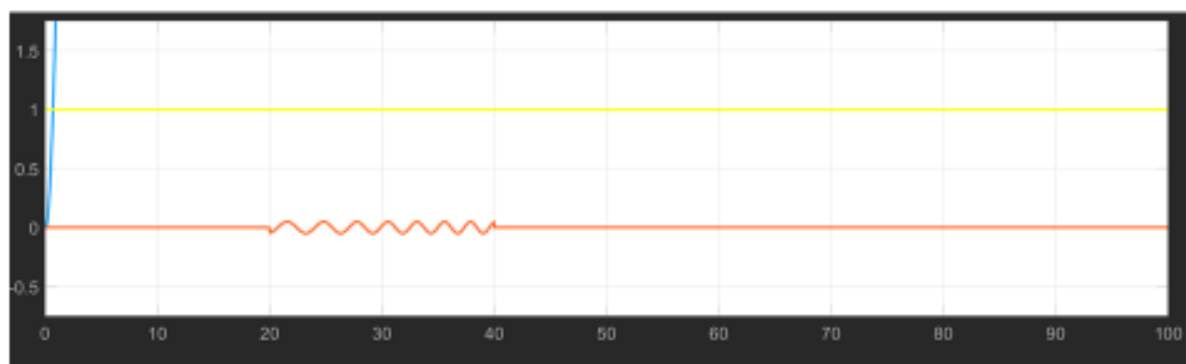
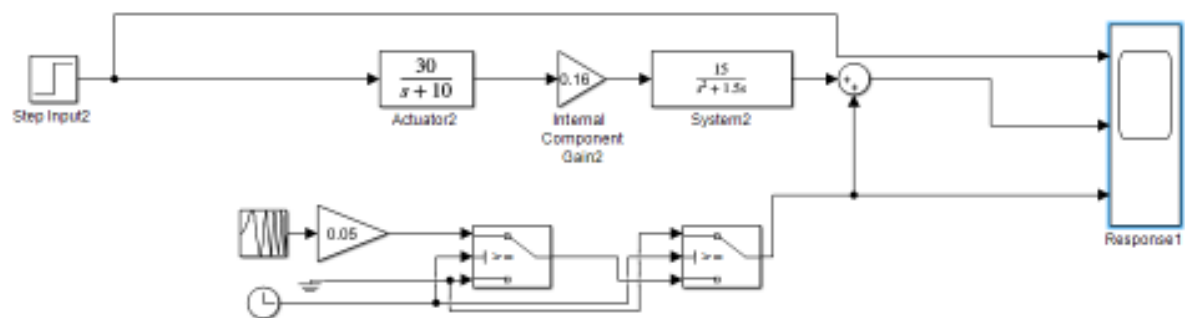
4. Closed-loop response with controller and without disturbance



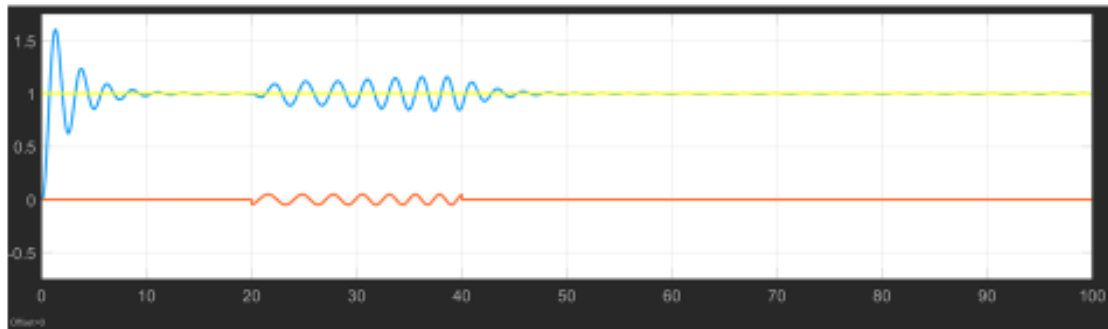
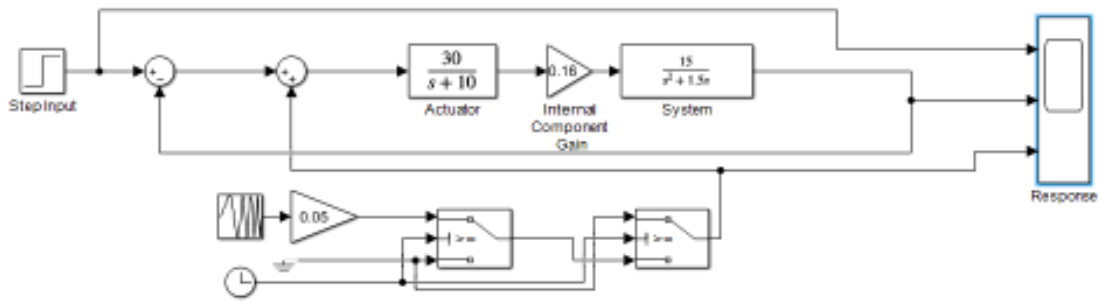
5. Open-loop response without controller and with disturbance (internal)



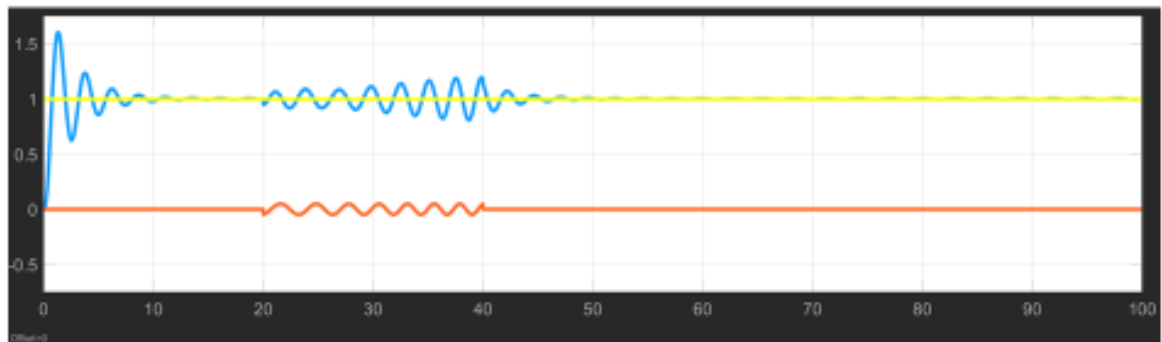
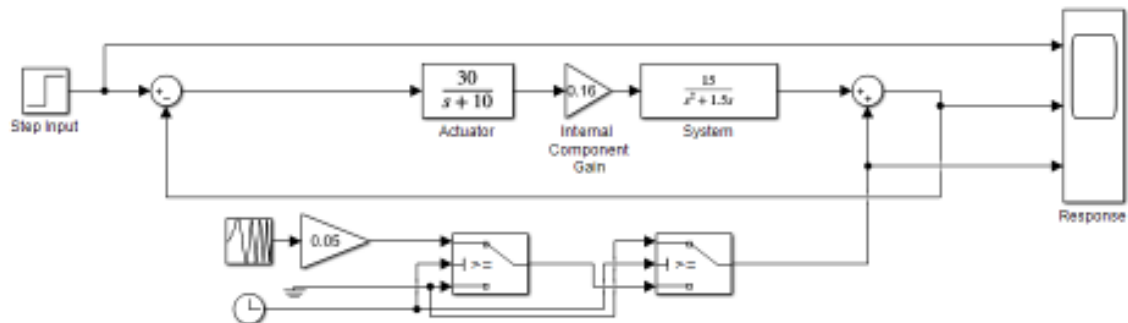
5. Open-loop response without controller and with disturbance (external)



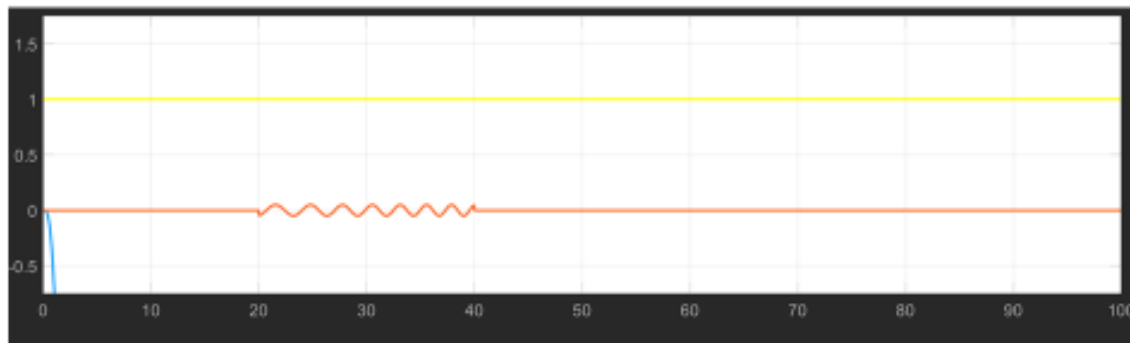
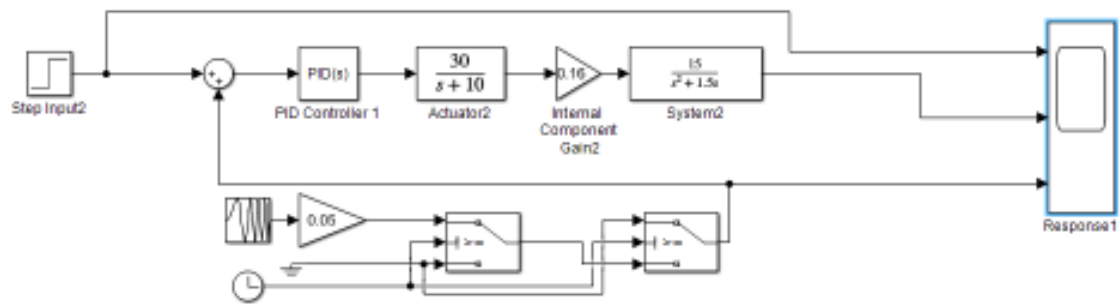
6. Closed-loop response without controller and with disturbance (internal)



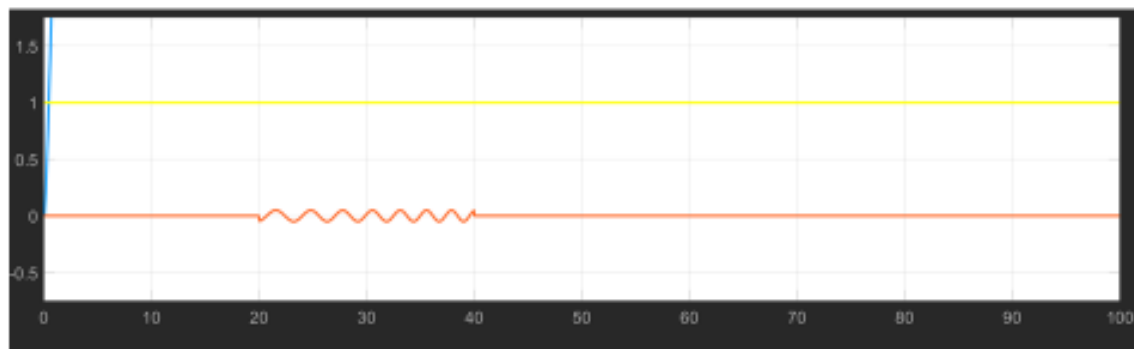
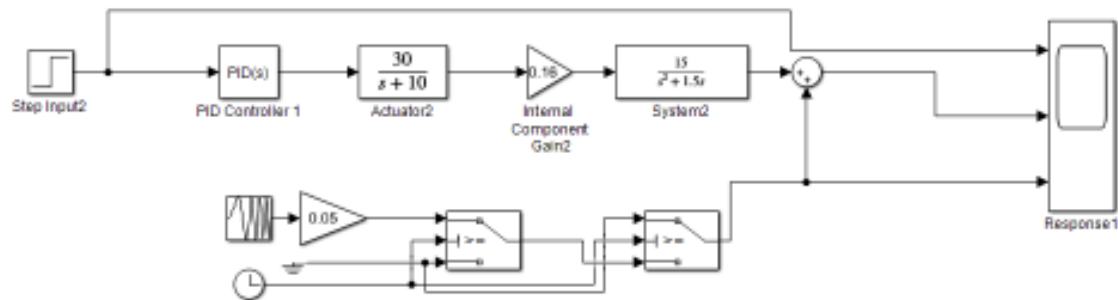
6. Closed-loop response without controller and with disturbance (external)



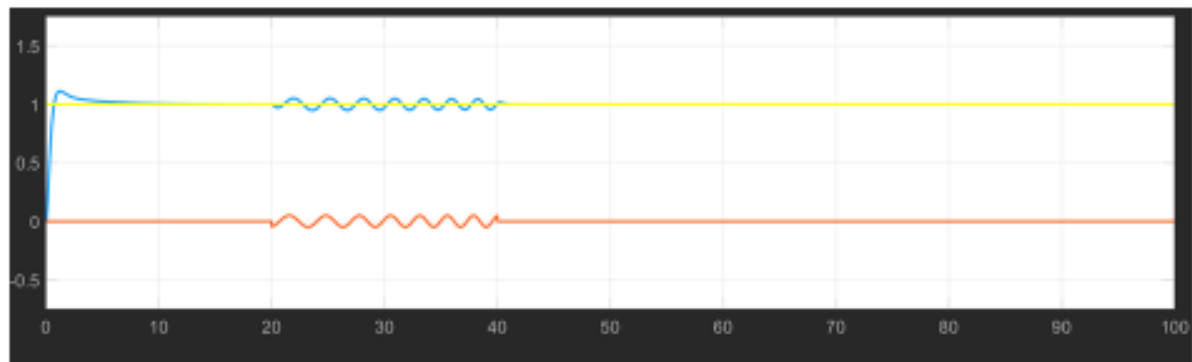
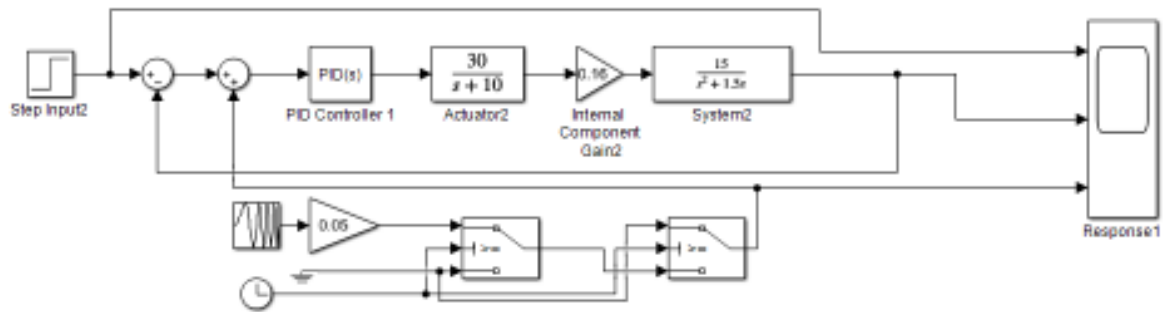
7. Open-loop response with controller and with disturbance (internal)



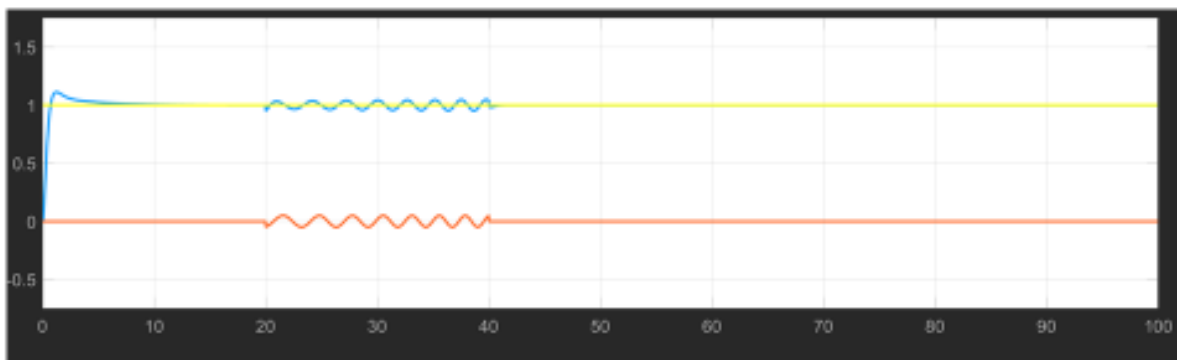
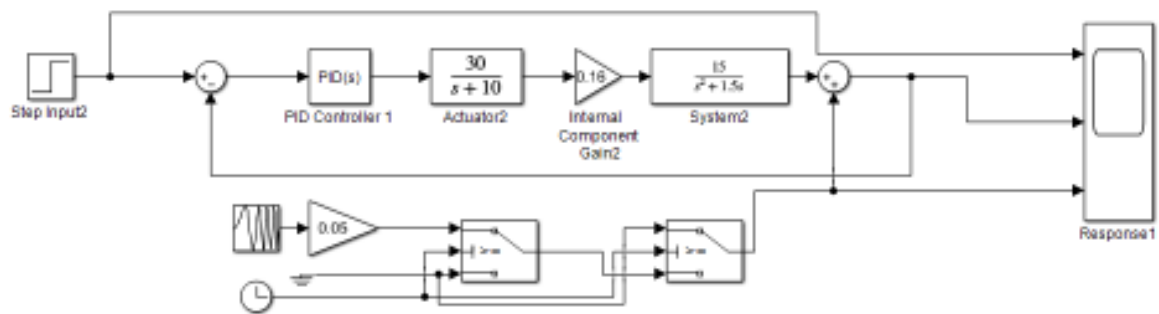
7. Open-loop response with controller and with disturbance (external)



8. Closed-loop response with controller and with disturbance (internal)



8. Closed-loop response with controller and with disturbance (external)



Expt. No.:8

Date:

State space analysis of a system.

AIM/OBJECTIVE:

To analyze state space modeling of system and design the controller for this.

APPARATUS /COMPONENTS REQUIRED:

S.No	Name of the components	Range	Quantity
1	MATLAB/Simulink		
2	Control system toolbox		

Theory:

Modelling

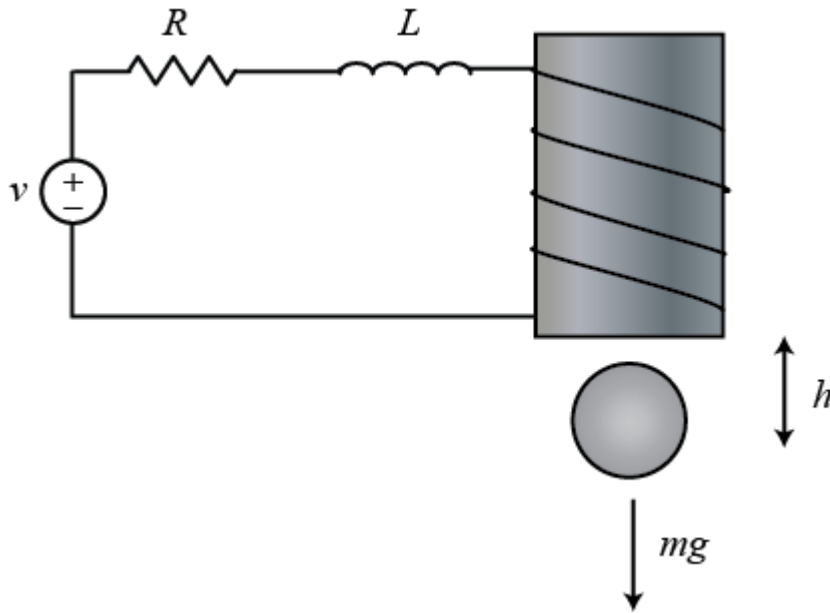
There are several different ways to describe a system of linear differential equations. For a SISO LTI system, the state-space form is given below:

$$(1) \frac{dx}{dt} = Ax + Bu$$

$$(2) y = Cx + Du$$

where $\mathbf{x}$ is an n by 1 vector representing the system's state variables, u is a scalar representing the input, and y is a scalar representing the output. The matrices A (n by n), B (n by 1), and C (1 by n) determine the relationships between the state variables and the input and output. Note that there are n first-order differential equations. A state-space representation can also be used for systems with multiple inputs and multiple outputs (MIMO), but we will primarily focus on single-input, single-output (SISO) systems in these tutorials.

To introduce the state-space control design method, we will use the magnetically suspended ball as an example. The current through the coils induces a magnetic force which can balance the force of gravity and cause the ball (which is made of a magnetic material) to be suspended in mid-air. The modeling of this system has been established in many control text books (including *Automatic Control Systems* by B. C. Kuo, the seventh edition).



The equations for the system are given by:

$$(3) \quad m \frac{d^2 h}{dt^2} = mg - \frac{K i^2}{h}$$

$$(4) \quad V = L \frac{di}{dt} + iR$$

where h is the vertical position of the ball, i is the current through the electromagnet, V is the applied voltage, m is the mass of the ball, g is the acceleration due to gravity, L is the inductance, R is the resistance, and K is a coefficient that determines the magnetic force exerted on the ball. For simplicity, we will choose values $m = 0.05$ kg, $K = 0.0001$, $L = 0.01$ H, $R = 1$ Ohm, $g = 9.81$ m/s². The system is at equilibrium (the ball is suspended in mid-air) whenever $h = K i^2 / mg$ (at which point $dh/dt = 0$). We linearize the equations about the point $h = 0.01$ m (where the nominal current is about 7 Amps) and obtain the linear state-space equations:

$$(5) \quad \frac{dx}{dt} = Ax + Bu$$

$$(6) \quad y = Cx + Du$$

where:

$$(7) \quad x = \begin{bmatrix} \Delta h \\ \Delta \dot{h} \\ \Delta i \end{bmatrix}$$

is the set of state variables for the system (a 3x1 vector), u is the deviation of the input voltage from its equilibrium value (ΔV), and y (the output) is the deviation of the height of the ball from its equilibrium position (Δh). Enter the system matrices into an m-file.

```
A = [ 0  1  0
      980 0 -2.8
      0  0 -100 ];
```

```
B = [ 0  
      0  
      100 ];
```

```
C = [ 1 0 0 ];
```

Stability

One of the first things we want to do is analyze whether the open-loop system (without any control) is stable. As discussed in the [Introduction: System Analysis](#) section, the eigenvalues of the system matrix, A , (equal to the poles of the transfer function) determine stability. The eigenvalues of the A matrix are the values of s that are solutions of $\det(sI - A) = 0$.

```
poles = eig(A)
```

```
poles =  
  
    31.3050  
  
   -31.3050  
  
  -100.0000
```

From inspection, it can be seen that one of the poles is in the right-half plane (i.e. has positive real part), which means that the open-loop system is unstable.

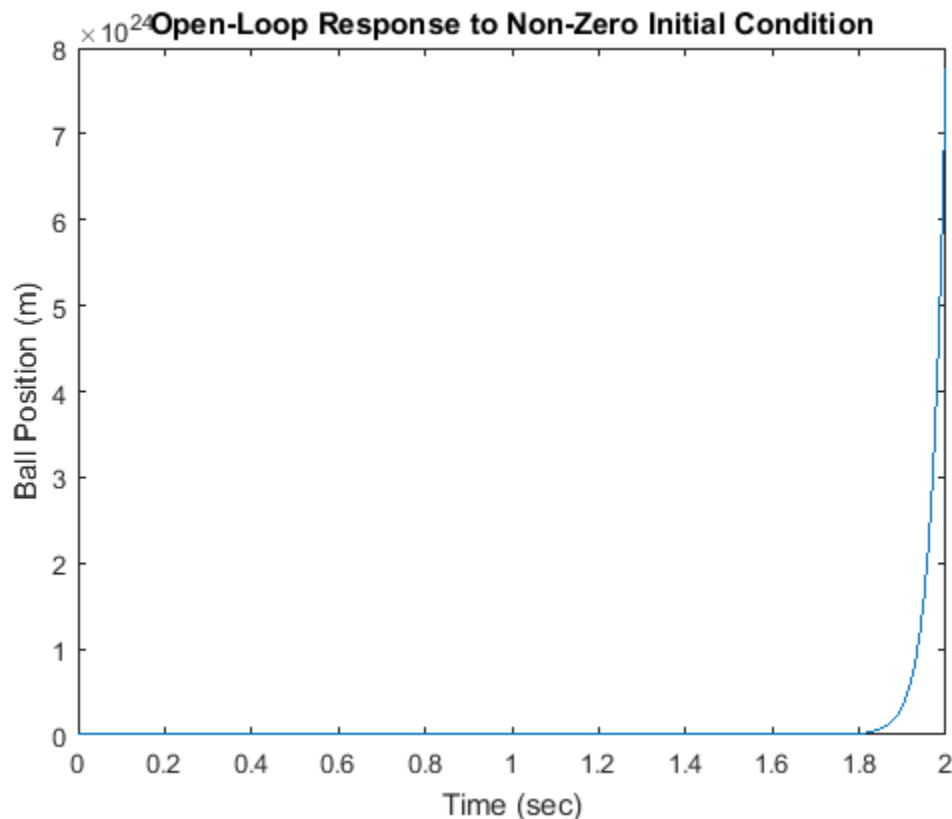
To observe what happens to this unstable system when there is a non-zero initial condition, add the following lines to your m-file and run it again:

```
t = 0:0.01:2;  
  
u = zeros(size(t));  
  
x0 = [0.01 0 0];  
  
  
sys = ss(A,B,C,0);
```

```
[y,t,x] = lsim(sys,u,t,x0);

plot(t,y)

title('Open-Loop Response to Non-Zero Initial Condition')
xlabel('Time (sec)')
ylabel('Ball Position (m)')
```



It looks like the distance between the ball and the electromagnet will go to infinity, but probably the ball hits the table or the floor first (and also probably goes out of the range where our linearization is valid).

Controllability and Observability

A system is **controllable** if there always exists a control input, $u(t)$, that transfers any state of the system to any other state in finite time. It can be shown that an LTI system is controllable if and only if its controllability matrix, $\mathcal{C}$, has full rank (i.e. if $\text{rank}(\mathcal{C}) = n$ where n is the number of states variables). The rank of the controllability matrix of an LTI model can be determined in MATLAB using the commands `rank(ctrb(A,B))` or `rank(ctrb(sys))`.

$$(8) \mathcal{C} = [B \ AB \ A^2B \ \dots \ A^{n-1}B]$$

All of the state variables of a system may not be directly measurable, for instance, if the component is in an inaccessible location. In these cases it is necessary to **estimate** the values of the unknown internal state variables using only the available system outputs. A system is **observable** if the initial state, $x(t_0)$, can be determined based on knowledge of the system input, $u(t)$, and the system output, $y(t)$, over some finite time interval $t_0 < t < t_f$. For LTI

systems, the system is observable if and only if the observability matrix, $\mathcal{O}$, has full rank (i.e. if $\text{rank}(\mathcal{O}) = n$ where n is the number of state variables). The observability of an LTI model can be determined in MATLAB using the command `rank(observ(A,C))` or `rank(observ(sys))`.

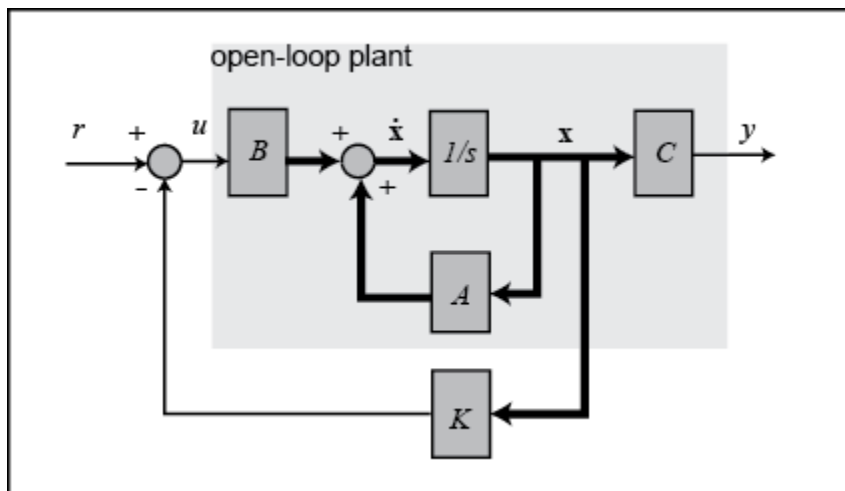
$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}$$

(9)

Controllability and observability are **dual** concepts. A system (A, B) is controllable if and only if a system (A', B') is observable. This fact will be useful when designing an observer, as we shall see below.

Control Design Using Pole Placement

Let's build a controller for this system using a pole placement approach. The schematic of a full-state feedback system is shown below. By full-state, we mean that all state variables are known to the controller at all times. For this system, we would need a sensor measuring the ball's position, another measuring the ball's velocity, and a third measuring the current in the electromagnet.



For simplicity, let's assume the reference is zero, $r = 0$. The input is then

$$(10) u = -Kx$$

The state-space equations for the closed-loop feedback system are, therefore,

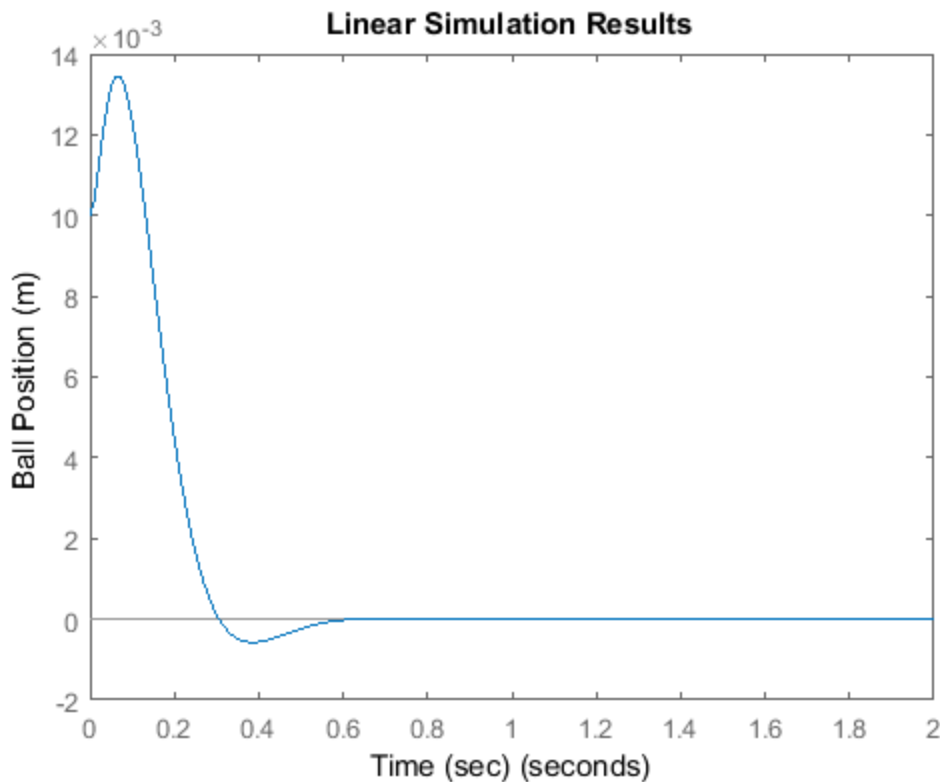
$$(11) \dot{x} = Ax + B(-Kx) = (A - BK)x$$

$$(12) y = Cx$$

The stability and time-domain performance of the closed-loop feedback system are determined primarily by the location of the eigenvalues of the matrix $(A - BK)$, which are equal to the closed-loop poles. Since the matrices A and BK are both 3×3 , there will be 3 poles for the system. By choosing an appropriate state-feedback gain matrix K , we can place these closed-loop poles anywhere we'd like (because the system is controllable). We can use the MATLAB function `place` to find the state-feedback gain, K , which will provide the desired closed-loop poles.

Before attempting this method, we have to decide where we want to place the closed-loop poles. Suppose the criteria for the controller were settling time < 0.5 sec and overshoot $< 5\%$, then we might try to place the two dominant poles at $-10 \pm 10i$ (at $\zeta = 0.7$ or 45 degrees with $\sigma = 10 > 4.6 \times 2$). The third pole we might place at -50 to start (so that it is sufficiently fast that it won't have much effect on the response), and we can change it later depending on what closed-loop behavior results. Remove the `lsim` command from your m-file and everything after it, then add the following lines to your m-file:

```
p1 = -10 + 10i;  
  
p2 = -10 - 10i;  
  
p3 = -50;  
  
  
K = place(A,B,[p1 p2 p3]);  
  
sys_cl = ss(A-B*K,B,C,0);  
  
  
lsim(sys_cl,u,t,x0);  
  
xlabel('Time (sec)')  
ylabel('Ball Position (m)')
```



From inspection, we can see the overshoot is too large (there are also zeros in the transfer function which can increase the overshoot; you do not explicitly see the zeros in the state-space formulation). Try placing the poles further to the left to see if the transient response improves (this should also make the response faster).

```
p1 = -20 + 20i;
```

```
p2 = -20 - 20i;
```

```
p3 = -100;
```

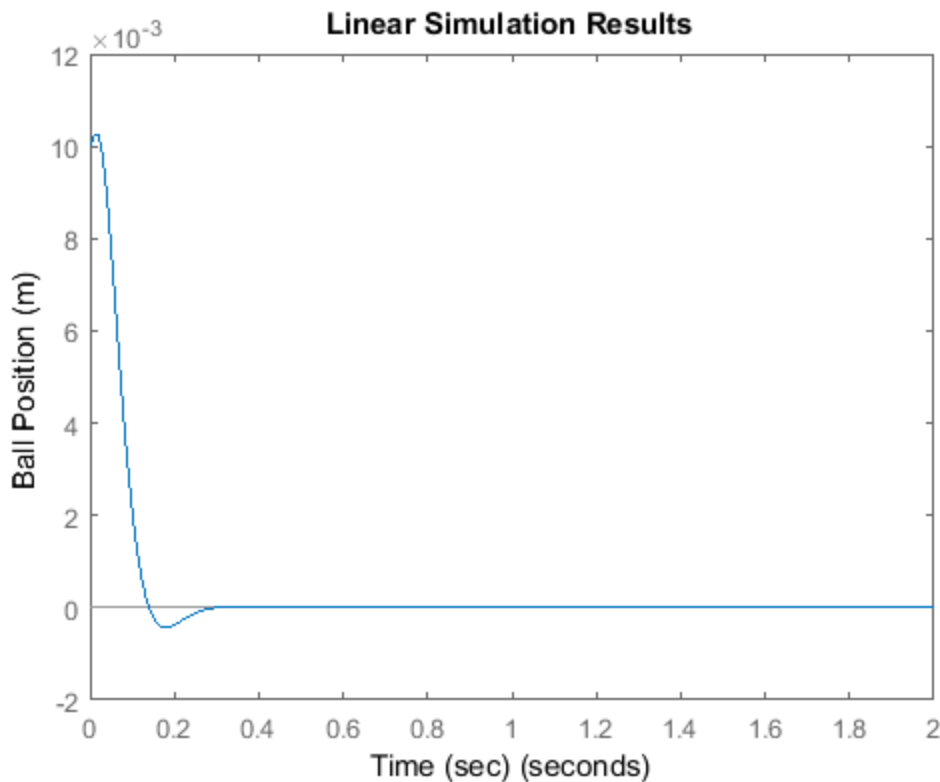
```
K = place(A,B,[p1 p2 p3]);
```

```
sys_cl = ss(A-B*K,B,C,0);
```

```
lsim(sys_cl,u,t,x0);
```

```
xlabel('Time (sec)')
```

```
ylabel('Ball Position (m)')
```



This time the overshoot is smaller. Consult your textbook for further suggestions on choosing the desired closed-loop poles.

Compare the control effort required (u) in both cases. In general, the farther you move the poles to the left, the more control effort is required.

Note: If you want to place two or more poles at the same position, place will not work. You can use a function called `acker` which achieves the same goal (but can be less numerically well-conditioned):

```
K = acker(A,B,[p1 p2 p3])
```

Introducing the Reference Input

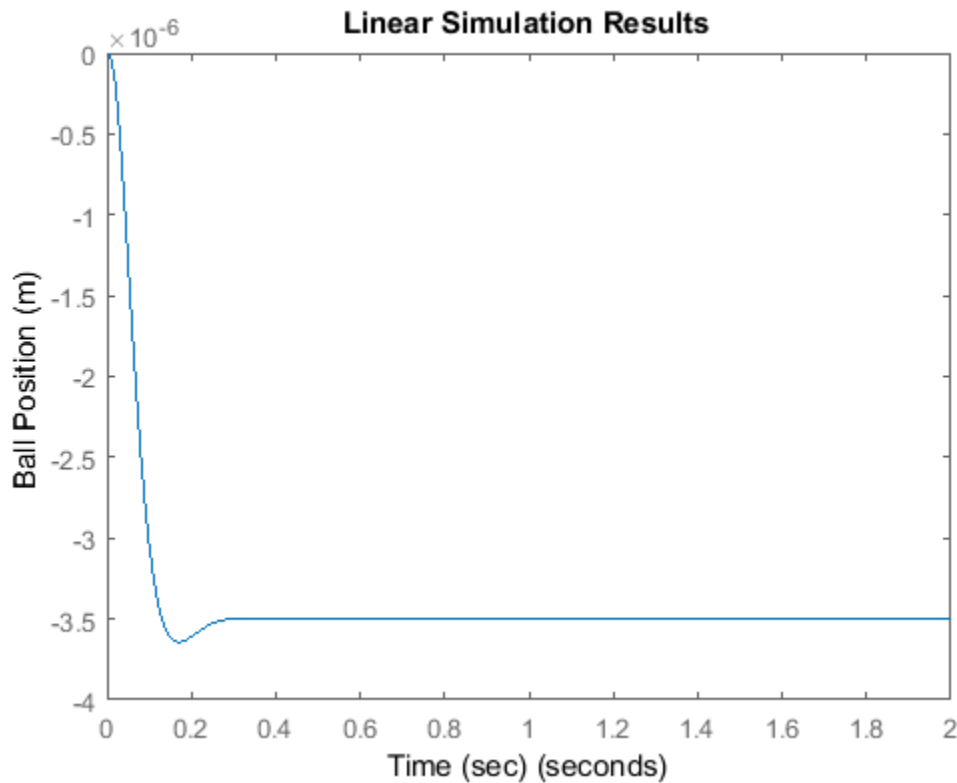
Now, we will take the control system as defined above and apply a step input (we choose a small value for the step, so we remain in the region where our linearization is valid). Replace `t`, `u`, and `lsim` in your m-file with the following:

```
t = 0:0.01:2;

u = 0.001*ones(size(t));

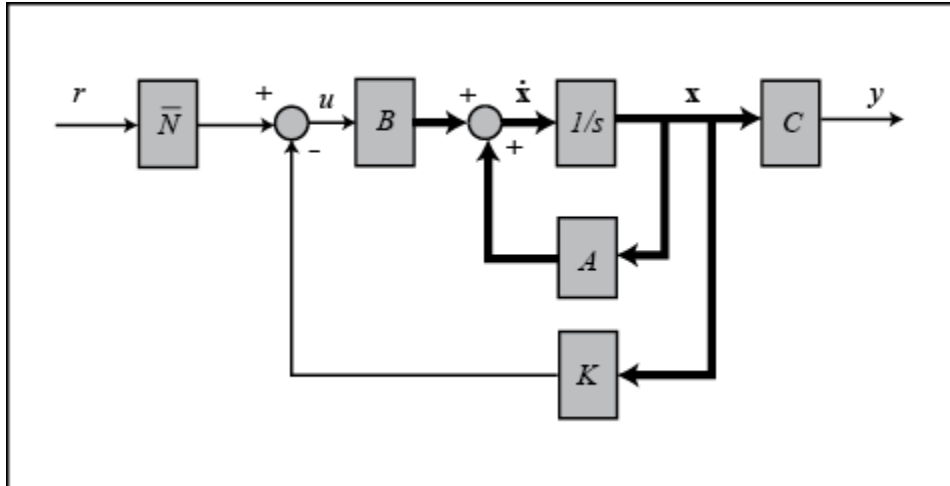
sys_cl = ss(A-B*K,B,C,0);
```

```
lsim(sys_cl,u,t);  
  
xlabel('Time (sec)')  
ylabel('Ball Position (m)')  
axis([0 2 -4E-6 0])
```



The system does not track the step well at all; not only is the magnitude not one, but it is negative instead of positive!

Recall the schematic above, we don't compare the output to the reference; instead we measure all the states, multiply by the gain vector K , and then subtract this result from the reference. There is no reason to expect that $K\mathbf{x}$ will be equal to the desired output. To eliminate this problem, we can scale the reference input to make it equal to $K\mathbf{x}$ in steady-state. The scale factor, $\bar{N}$, is shown in the following schematic:



We can calculate $\bar{N}$ within MATLAB employing the function `rscale` (place the following line of code after `K = ...`).

```
Nbar = rscale(sys,K)
```

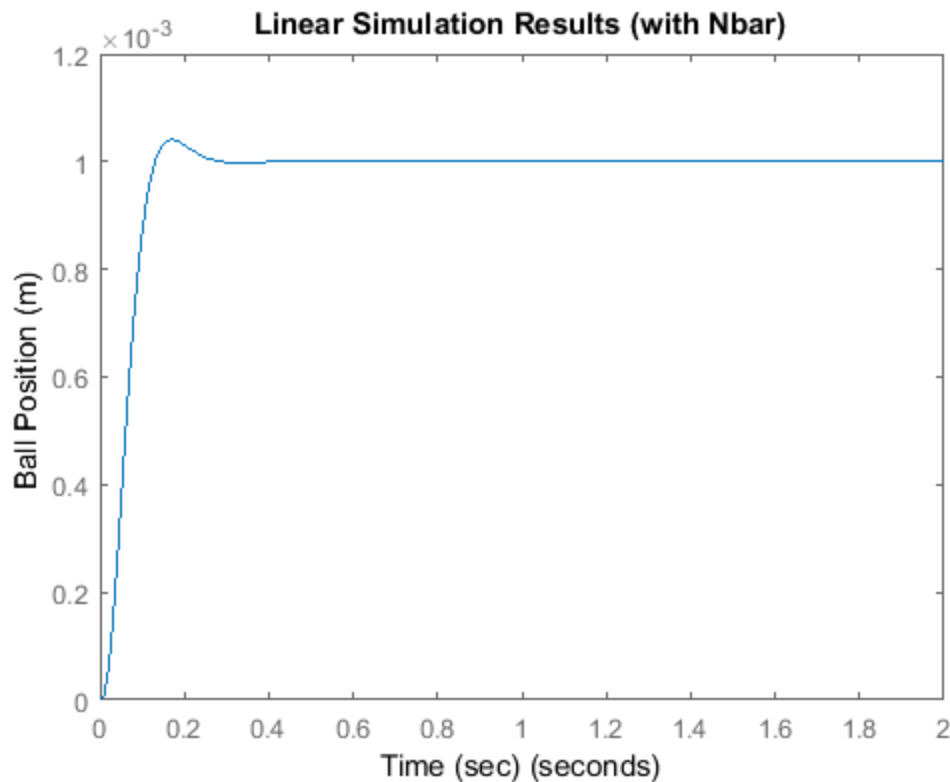
Nbar =

-285.7143

Note that this function is not standard in MATLAB. You will need to download it here, rscale.m, and save it to your current workspace. Now, if we want to find the response of the system under state feedback with this scaling of the reference, we simply note the fact that the input is multiplied by this new factor, $\bar{N}$:

```
lsim(sys_cl,Nbar*u,t)
```

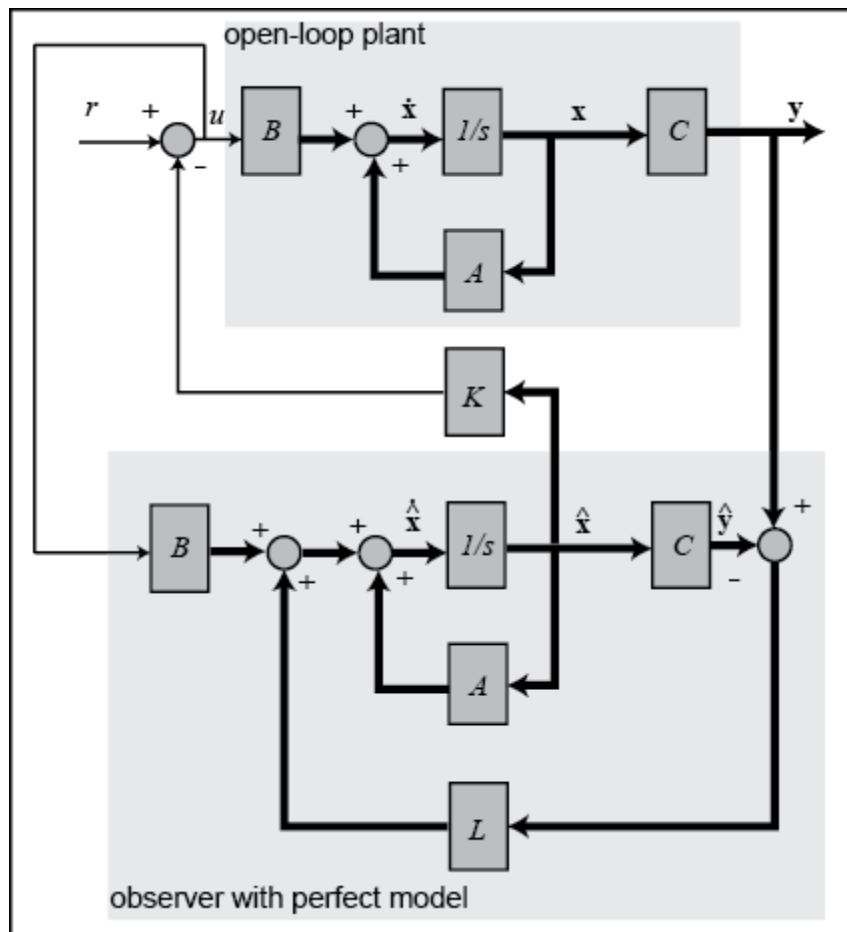
```
title('Linear Simulation Results (with Nbar)')
xlabel('Time (sec)')
ylabel('Ball Position (m)')
axis([0 2 0 1.2*10^-3])
```



and now a step can be tracked reasonably well. Note, our calculation of the scaling factor requires good knowledge of the system. If our model is in error, then we will scale the input an incorrect amount. An alternative, similar to what was introduced with PID control, is to add a state variable for the integral of the output error. This has the effect of adding an integral term to our controller which is known to reduce steady-state error.

Observer Design

When we can't measure all state variables $\mathbf{x}$ (often the case in practice), we can build an **observer** to estimate them, while measuring only the output $y = C\mathbf{x}$. For the magnetic ball example, we will add three new, estimated state variables ($\hat{\mathbf{x}}$) to the system. The schematic is as follows:



The observer is basically a copy of the plant; it has the same input and almost the same differential equation. An extra term compares the actual measured output y to the estimated output $\hat{y} = C\hat{x}$; this will help to correct the estimated state $\hat{x}$ and cause it to approach the values of the actual state x (if the measurement has minimal error).

$$(13) \dot{\hat{x}} = A\hat{x} + Bu + L(y - \hat{y})$$

$$(14) \hat{y} = C\hat{x}$$

The error dynamics of the observer are given by the poles of $A - LC$.

$$(15) \dot{e} = \dot{x} - \dot{\hat{x}} = (A - LC)e$$

First, we need to choose the observer gain L . Since we want the dynamics of the observer to be much faster than the system itself, we need to place the poles at least five times farther to the left than the dominant poles of the system. If we want to use place, we need to put the three observer poles at different locations.

$$\text{op1} = -100;$$

$$\text{op2} = -101;$$

$$\text{op3} = -102;$$

Because of the duality between controllability and observability, we can use the same technique used to find the control matrix by replacing the matrix B by the matrix C and taking the transposes of each matrix:

```
L = place(A',C',[op1 op2 op3]);
```

The equations in the block diagram above are given for the estimate $\hat{\mathbf{x}}$. It is conventional to write the combined equations for the system plus observer using the original state equations plus the estimation error: $\mathbf{e} = \mathbf{x} - \hat{\mathbf{x}}$. We use the estimated state for feedback, $\mathbf{u} = -\mathbf{K}\hat{\mathbf{x}}$, since not all state variables are necessarily measured. After a little bit of algebra (consult your textbook for more details), we arrive at the combined state and error equations for full-state feedback with an observer.

```
At = [ A-B*K      B*K
      zeros(size(A))  A-L*C ];
```

```
Bt = [ B*Nbar
      zeros(size(B)) ];
```

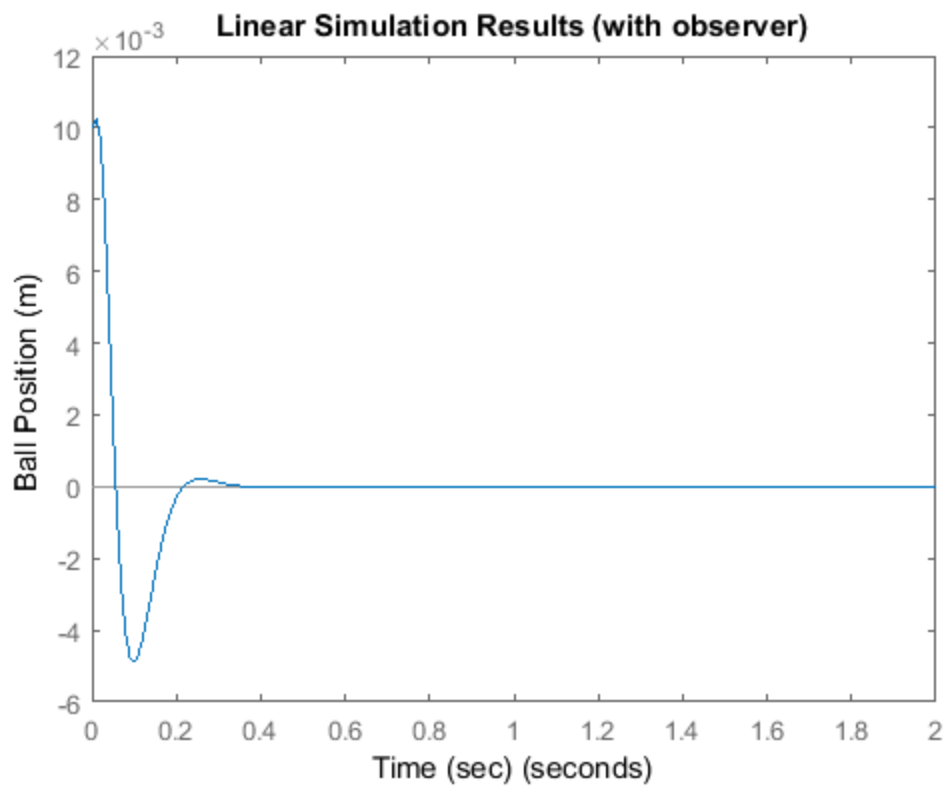
```
Ct = [ C  zeros(size(C)) ];
```

To see how the response to a non-zero initial condition with no reference input appears, add the following lines into your m-file. Here we will assume that the observer begins with an initial estimate equal to zero, such that the initial estimation error is equal to the initial state vector, $\mathbf{e} = \mathbf{x}$.

```
sys = ss(At,Bt,Ct,0);

lsim(sys,zeros(size(t)),t,[x0 x0]);

title('Linear Simulation Results (with observer)')
xlabel('Time (sec)')
ylabel('Ball Position (m)')
```



Responses of all the state variables are plotted below. Recall that `lsim` gives us $\mathbf{x}$ and $\mathbf{e}$; to get $\hat{\mathbf{x}}$, we need to compute $\mathbf{x} - \mathbf{e}$.

```
t = 0:1E-6:0.1;

x0 = [0.01 0.5 -5];

[y,t,x] = lsim(sys,zeros(size(t)),t,[x0 x0]);

n = 3;

e = x(:,n+1:end);

x = x(:,1:n);

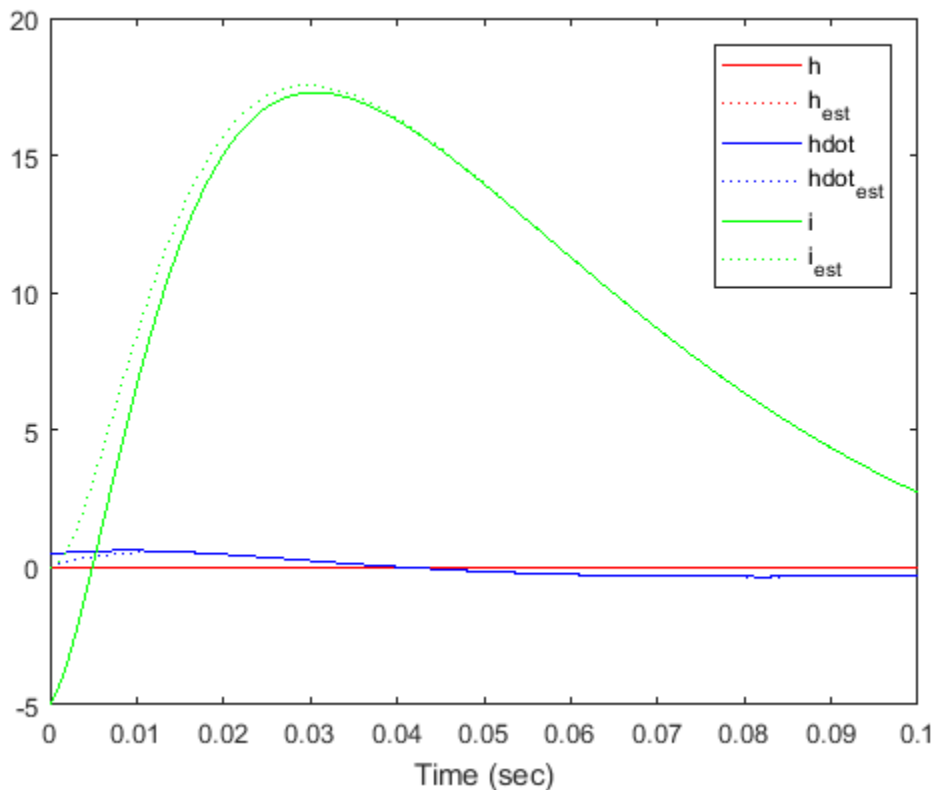
x_est = x - e;
```

```

% Save state variables explicitly to aid in plotting
h = x(:,1); h_dot = x(:,2); i = x(:,3);
h_est = x_est(:,1); h_dot_est = x_est(:,2); i_est = x_est(:,3);

plot(t,h,'-r',t,h_est,':r',t,h_dot,'-b',t,h_dot_est,':b',t,i,'-g',t,i_est,':g')
legend('h','h_{est}','hdot','hdot_{est}','i','i_{est}')
xlabel('Time (sec)')

```



From the above, we can see that the observer estimates converge to the actual state variables quickly and track the state variables well in steady-state.